Licenciatura em Engenharia Informática

Project II

Beta Breaker

Project Report

Miguel Leitão Cardoso

a22207062

miguelcardoso499@proton.me

Advisor: Hugo Barbosa

External Entity: N/A

Department of Computer Science Engineering and Information Systems

Universidade Lusófona, Centro Universitário do Porto (CUP)

2025/2026

www.ulusofona.pt

Copyright

Acknowledgments

I would like to thank Professor Hugo Barbosa for his guidance and availability throughout this project, and my parents for their patience and continued support.

Resumo

Os ginásios de escalada indoor dependem da rotação frequente de vias e do envolvimento da comunidade para reter os seus membros, mas a informação sobre vias, o conhecimento de beta e o acompanhamento da progressão permanecem, em grande parte, informais. Este projeto apresenta o Beta Breaker, uma aplicação móvel multiplataforma que liga escaladores e ginásios através da descoberta de vias, registo de ascensões, partilha de vídeos de beta e gamificação. Construído com React Native e Expo sobre um backend Supabase, o sistema utiliza Row Level Security do PostgreSQL como fronteira de autorização, eliminando a necessidade de um servidor API dedicado. A aplicação suporta fluxos de leitura de código QR para aceder a vias, funcionamento offline através de uma fila de ações em SQLite, quadros de competição em tempo real e uma subscrição Pro gerida por compras in-app nativas. Foi seguida uma metodologia rigorosa de Test-Driven Development ao longo de toda a implementação, produzindo 1 975 testes automatizados (1 480 unitários e 495 de integração) em 83 passos de desenvolvimento concluídos. Em março de 2026, 19 das 21 fases planeadas estão totalmente implementadas, cobrindo todos os requisitos funcionais e fornecendo uma aplicação completa pronta para testes end-to-end e submissão nas lojas.

Palavras-chave: escalar; ginásio; gamificação; beta; mobile

Abstract

Indoor climbing gyms rely on frequent route rotation and community engagement to retain members, yet route information, beta knowledge, and progression tracking remain largely informal. This project presents Beta Breaker, a cross-platform mobile application that connects climbers and gyms through route discovery, ascent logging, beta video sharing, and gamification. Built with React Native and Expo on a Supabase backend, the system uses PostgreSQL Row Level Security as its authorization boundary, eliminating the need for a custom API server. The application supports QR-based scan-to-route flows, offline operation via an SQLite action queue, real-time competition scoreboards, and a Pro subscription tier managed through native in-app purchases. A strict Test-Driven Development methodology was followed throughout the implementation, producing 1 975 automated tests (1 480 unit and 495 integration) across 83 completed development steps. As of March 2026, 19 of 21 planned phases are fully implemented, covering all functional requirements and providing a feature-complete application ready for end-to-end testing and store submission.

Keywords: climbing; gym; gamification; beta; mobile

Contents

List of Figures

List of Tables

List of Acronyms

API Application Programming Interface

B2B2C Business-to-Business-to-Consumer

CORS Cross-Origin Resource Sharing

CSV Comma-Separated Values

GDPR General Data Protection Regulation

HTTPS Hypertext Transfer Protocol Secure

IAP In-App Purchase

IDE Integrated Development Environment

MVP Minimum Viable Product

NFC Near Field Communication

PDF Portable Document Format

PII Personally Identifiable Information

QR Quick Response (code)

RLS Row Level Security

SRS Software Requirements Specification

SWOT Strengths, Weaknesses, Opportunities, Threats

TLS Transport Layer Security

TOC Table of Contents

UGC User-Generated Content

1 Introduction

1.1 Context

The project consists of developing a mobile application to support the digitalization and gamification of indoor climbing gyms. The application will allow climbers to view information about each climbing route, such as grade, wall location, and skills emphasized (e.g., finger strength, technique, dynamic movement, footwork, endurance). Users will also be able to submit beta videos, leave comments, and review routes to help others progress.

To enhance motivation, the app will include gamification [2] features such as achievements (badges for completing challenges, milestone ascents, or gym-specific objectives) and leaderboards (ranking climbers based on performance, number of routes completed, or difficulty level). Leaderboards will be complemented with prizes to reward top performers and stimulate continuous participation.

Additionally, the app will allow climbers to provide structured feedback to route setters, enabling gyms to collect insights about route quality, difficulty perception, and overall user experience.

The main goal is to create a community-driven platform that increases engagement with climbing gyms, motivates climbers through interactive features, and provides a structured way to analyze training and route styles.

Indoor climbers often lack a unified way to access up-to-date route information, beta videos, and meaningful progression tracking inside their gyms. Gyms also need a simple method to manage their routes and receive feedback from the climbers. Beta Breaker addresses these issues by providing climbers with clear route information, shared beta, achievements, and leaderboards that encourage consistent participation, while giving gyms a modern digital layer to manage their setting and community.

Beta Breaker is a platform that connects climbers and gyms around shared climbing experiences. It helps gyms publish and manage their indoor routes and community, while giving climbers a simple way to discover, log, and learn from their climbs. The product emphasizes fairness, meaningful feedback, and clear insight into progression, supporting both everyday sessions and gym-led initiatives such as seasons and events.

Product name: Beta Breaker

Vision statement. Create the go-to mobile platform that connects climbers and gyms through route discovery, beta sharing, and light-weight gamification. Beta Breaker motivates climbers to progress, helps gyms understand their community, and gives route setters actionable feedback to improve route quality.

Solution. A mobile app (iOS/Android) where climbers can browse routes, watch/upload beta videos, leave comments and ratings, and participate in challenges, achievements, and leaderboards. Gyms and route setters receive summarized feedback and simple analytics on route reception and difficulty perception.

Primary users. (1) Climbers who want to progress and engage with the community; (2) Route setters / gyms who want structured feedback and engagement tools.

1.2 Motivation

Indoor climbing is highly social, but knowledge about routes (beta, perceived difficulty, style) is fragmented across word-of-mouth and social media. Gyms lack structured, privacy-respectful ways to engage customers and collect feedback; climbers lack a single place to log, learn, and stay motivated.

The problem can be decomposed into the following sub-problems:

- **Identity & Accounts:** registration, profile, privacy and safety.
- **Gyms & Areas:** gym directory, sectors, social links.
- **Routes Catalog:** route model, QR/NFC IDs, user-driven tags and feedback.
- **Logging & Sessions:** quick logs, session start/end, summaries.
- **Analytics:** grade conversions, personalized suggestions (Pro).
- **Gamification:** badges, streaks, optional rank model.
- **Social:** leaderboards, feedback, reporting, video verification policy.
- **Admin Web:** audit logs and operations.
- **Monetization:** Pro subscriptions and gym billing console.

1.3 Objectives

1.3.1 Business Objectives (Project Charter)

The business objectives of this project are:

- Support climbing gyms in offering a modern digital service that complements their physical routes.
- Increase customer engagement and retention by gamifying the climbing experience.
- Provide climbers with tools to track progress, exchange knowledge, and connect with the community.
- Motivate climbers through gamification features such as achievements, challenges, leaderboards, and prizes.

- Strengthen the visibility of climbing gyms through digital integration and user-generated content.
- Provide route setters with valuable feedback from climbers, helping them adjust and improve future routes.

1.3.2 Business Goals (SRS)

- Increase climber engagement and retention through scan-to-log flows and rich feedback.
- Provide gyms with verifiable leaderboards, operational visibility, and simple billing.
- Enable fair progression by crowd-sourced tags and optional video verification.
- Monetize via Pro subscriptions (consumers) and per-active-user billing (gyms).

1.3.3 Project Scope

In Scope (MVP).

- **Route catalog:** Browse/search routes by grade, wall/sector, style/tags (e.g., finger strength, dynamic movement, footwork, endurance).
- **Beta and reviews:** View, upload, and report beta videos (basic moderation tools); leave comments and 1–5 rating plus perceived grade.
- **Gamification:** Achievements (e.g., first of grade, weekly streaks), simple leaderboards (per gym, per period), and optional prizes managed by gyms.
- **Feedback to setters:** Structured form per route (quality, grade perception, style markers); aggregated summaries and trends for gyms.
- **Accounts & privacy:** Email/social login, profile, basic notification settings, GDPR-aligned consent screens.
- **Admin (lightweight):** Gym/route management and review moderation (web or mobile admin views).

Out of Scope. Advanced training plans, wearables integration, Elo-like rank systems, and outdoor crag support.

1.3.4 Key Deliverables

- Lo-fi & Hi-fi prototypes with core user flows (browse, beta, feedback, achievements).
- Functional MVP implementing the features listed in “In Scope (MVP)”.
- Pilot deployment with seeded routes for at least one gym and a small user group.
- Documentation (requirements, architecture, test plan, project report) and a final presentation.

1.3.5 Quality Attributes

- **Usability:** Clear navigation, short paths to core actions (watch beta, log feedback).
- **Performance:** Route list loads in < 2 s on average Wi-Fi; videos stream with adaptive quality.
- **Security & privacy:** Least-privilege data access; user control over content and account data.
- **Maintainability:** Modular architecture; documented APIs and data models.

1.3.6 Glossary of Terms

Route/Problem
: A climb in the gym (boulder, top-rope, lead).

QR/NFC Tag
: Physical tag to open the route page in-app.

Beta
: Strategy or movement sequence to complete a route.

Verification Video
: Video evidence attached to a send for leaderboard validity.

Pro Tier
: Paid consumer plan with analytics, unlimited beta views, and 3 profile badges.

1.4 Document Structure

This report is organized as follows. Chapter 1 presents the context, motivation, identified problem, and the project objectives. Chapter 2 discusses the relevance and feasibility of the solution, including a review of existing solutions, the proposed added value, and the identified business opportunity. Chapter 3 focuses on specification and modelling, covering functional and non-functional requirements, the data model (ER diagram), and the interface prototypes. Chapter 4 describes the developed solution, including its architecture, technologies, and components. Chapter 5 covers the testing and validation approach, tools, and coverage. Chapter 6 presents the development methodology, work plan, and progress analysis. Chapter 7 reports test results and requirements compliance. Chapter 8 summarizes the conclusions and outlines future work.

2 Relevance and Feasibility

2.1 Relevance

Indoor climbing gyms continue to grow and increasingly rely on community engagement and fresh route setting to retain members. However, route information, style perception, and beta knowledge are still largely informal and fragmented. A platform that is tightly connected to the physical gym environment and its route lifecycle can improve the experience for climbers while providing gyms with structured feedback and actionable insights.

2.2 Feasibility

The project is designed as an MVP-first solution with an incremental delivery strategy. The scope is controlled to avoid complex payment flows and heavy analytics at this stage, focusing instead on core user value: route discovery, logging, feedback, and lightweight gamification. Technical feasibility is supported by a standard client–server architecture and commonly used technologies appropriate for iterative development.

2.2.1 SWOT Analysis

Strengths

- Clear niche focus (indoor climbing) with community-driven content.
- Gamification aligned with climbers' intrinsic motivation and progression.
- Actionable, structured feedback loop for route setters and gyms.
- Lightweight tech stack suitable for a student project with iterative delivery.

Weaknesses

- Reliance on user-generated content (requires seeding and moderation).
- Limited resources for content review, partnerships, and support.
- Potential data sparsity if a gym's community adopts slowly (network effects).
- Early analytics are basic; may not meet advanced gym expectations.

Opportunities

- Rapid growth of indoor climbing and new gyms seeking differentiation.
- Partnerships with gyms for events, challenges, and prize sponsorships.
- Extensions to training insights (style profiling, personalized goals).
- Integrations with existing gym systems (check-in, route databases) and social platforms.

Threats

- Competing apps or gyms' own solutions reducing adoption incentives.
- Content/privacy incidents (e.g., inappropriate uploads) harming trust.
- Platform policy changes (app stores) affecting distribution or features.
- Low engagement if gamification is mis-tuned or prizes are inconsistent.

2.2.2 Cost Structure and Financial Viability

The project relies on managed cloud services with generous free tiers, keeping the MVP operational cost near zero during development and early adoption. Table 2.1 summarizes the infrastructure costs at three usage levels: development (current), pilot (1 gym, ∼200 active users), and growth (5 gyms, ∼1 000 active users).

Table 2.1: Monthly infrastructure costs by usage tier.

Service	**Free Tier Limit**	**Dev**	**Pilot**	**Growth**
Supabase (DB, Auth, Storage)	500 MB DB, 1 GB storage, 50K MAU	€0	€0	€25[a]
Cloudinary (video hosting)	25 GB storage, 25 GB bandwidth/mo	€0	€0	€0[b]
EAS Build (Expo)	30 builds/mo	€0	€0	€0
Sentry (error tracking)	5K errors/mo	€0	€0	€0
Apple Developer Program	—	€99/yr	€99/yr	€99/yr
Google Play Developer	—	€25 once	—	—
Total (monthly)		**€0**	**∼€8**	**∼€33**

[a] Pro plan ($25/mo) required when DB exceeds 500 MB or concurrent connections exceed 200.

[b] At 1 000 users with ∼50 video uploads/month (∼150 MB each), storage stays within free tier for ∼12 months.

Revenue enters from two streams. On the consumer side, the Pro subscription is priced at €4.99/month (or €49.99/year), offering analytics, unlimited beta video views, and 3 profile badges. On the gym side, the B2B model charges €0.50 per active gym-linked user per month, providing route management, feedback dashboards, and leaderboard administration.

Table 2.2 estimates the break-even point under conservative adoption assumptions.

At pilot scale (1 gym, 200 users), the platform becomes profitable from the first month of monetization, since infrastructure costs remain within or near free-tier limits. The low operational cost is a direct consequence of the Supabase-first architecture: no custom servers to maintain, no DevOps overhead, and pay-as-you-grow pricing from all service providers. At the growth stage (5 gyms, 1 000 users, 50 Pro subscribers), monthly revenue rises to approximately €750 against €33 in costs, yielding a healthy margin even before accounting for Apple/Google's 15–30% commission on in-app purchases.

The main financial risk is slow gym adoption, which would delay B2B revenue. This is mitigated by the near-zero cost base: the platform can operate indefinitely on free tiers during the acquisition phase without generating losses.

Table 2.2: Break-even estimate under conservative adoption.

Parameter	**Value**
Active users (pilot: 1 gym)	200
Pro conversion rate	5%
Pro subscribers	10
Monthly Pro revenue (10 × €4.99)	€49.90
Gym B2B revenue (200 × €0.50)	€100.00
Monthly revenue	**€149.90**
Monthly infrastructure cost (pilot)	€8
Monthly margin	**€141.90**

2.3 Comparative Analysis with Existing Solutions

2.3.1 Existing Solutions

A preliminary market scan shows tools that address only parts of the problem: (i) generic training/logbook applications, usually centered on the user and not tied to a gym's live route set; (ii) internal or informal gym route management approaches (e.g., boards/spreadsheets) that lack structured community feedback; and (iii) social platforms that enable sharing, but without a direct connection to routes, setters, seasons, and aggregated gym analytics.

Overall, no direct competitor was identified that offers an end-to-end product combining: gym route management, in-gym route discovery, ascent logging, structured feedback, optional video verification for fair leaderboards, and aggregated style/statistics at both route and gym level.

2.3.2 Benchmarking Analysis

Table 2.3 compares Beta Breaker with four established climbing applications across the core feature categories identified in the requirements analysis. Data was collected from official product pages and app store descriptions.

Table 2.3: Feature comparison with existing climbing applications.

Feature	**Beta Breaker**	**KAYA**	**Pebble**	**Vertical-Life**	**TopLogger**
Route catalog	✓	✓	✓	✓	✓
QR scan-to-route	✓	✓	✓	✓	?
Ascent logging	✓	✓	✓	✓	✓
Beta video sharing	✓	✓	✓	✓	?
Gamification (badges)	✓	✓	✓	✓	?
Leaderboards	✓	✓	✓	✓	✓

Feature	Beta Breaker	KAYA	Pebble	Vertical-Life	TopLogger
Setter feedback	✓	✓	✓	✓	Partial
Offline support	✓	Pro	?	Premium	?
Competitions / events	✓	✓	✓	✓	✓
Video verification	✓	—	—	—	—
Style taxonomy	✓	—	—	—	Partial
Climber price	Freemium	$59.99/yr	Free	€59.88/yr	Free
Gym price	€0.50/user/mo	~$100/mo	15% comp fee	$1 080/yr	$89.95/mo[a]

✓ = Confirmed ? = Unverified — = Not offered Partial = Limited implementation

[a] Free for gyms with fewer than 35 users.

All four competitors offer core route browsing and ascent logging, and three of the four support QR scanning and beta videos. However, none of the surveyed solutions offers video verification for leaderboard sends or a controlled style taxonomy with crowd-sourced consensus. Beta Breaker's offline support is included in the free tier, whereas KAYA and Vertical-Life reserve it for paying users. The gym pricing model (€ 0.50 per active user per month) is designed to scale linearly and remain competitive with the fixed-fee models of KAYA and Vertical-Life.

2.4 Innovation Proposal and Added Value

The main innovation lies in merging the climber experience and gym operations into one connected platform. Research suggests that gamification can positively influence engagement and motivation in digital services [4]. Each ascent can generate value for both sides: climbers receive a richer, more motivating experience (history, progress, community beta, and gamification), while gyms and setters receive structured feedback, difficulty perception signals, and usage trends.

Key added-value points include: (i) an in-gym, scan-to-route flow that reduces friction; (ii) a controlled style taxonomy and crowd-based style characterization that can be aggregated into route and gym profiles; (iii) lightweight gamification (badges, seasons, leaderboards, prizes) to improve retention; and (iv) a data model built to support aggregated statistics without constant expensive recomputation.

2.5 Business Opportunity

The business opportunity is primarily supported by the lack of direct competition identified for a gym-oriented, integrated solution. The product naturally fits a B2B2C positioning:

gyms adopt the platform to strengthen community engagement and improve route lifecycle visibility, while climbers gain a centralized and motivating experience.

A plausible monetization path is subscription/licensing per gym (tiered by feature modules or active users), optionally complemented by premium consumer features. Market entry can start with a small number of pilot gyms to validate adoption, refine the user experience, and then expand to other gyms with similar needs.

2.6 Business Risks

The following risks could affect delivery or adoption of the MVP. Each item includes a primary mitigation.

- **Cold start (adoption/content).** Few early users or videos reduce value. *Mitigation:* seed routes and beta with a pilot gym; launch a short challenge to generate initial content.
- **Content safety.** Inappropriate or IP-infringing uploads harm trust. *Mitigation:* clear guidelines, report/takedown flow, uploader ownership affirmation.
- **Video performance/cost.** Upload/streaming can be slow or exceed free tiers. *Mitigation:* client-side compression, duration/resolution caps, lazy loading, usage monitoring.
- **Scope creep and schedule slip.** Extra features delay MVP. *Mitigation:* milestone feature freeze, time-boxed sprints, “later” backlog.
- **Security & privacy.** Weak auth or poor consent risks data exposure/GDPR issues. *Mitigation:* managed auth, HTTPS, rate limits, least-privilege, consent + delete/export.
- **Third-party dependence.** Changes in auth/storage/CDN pricing or APIs. *Mitigation:* pin versions, thin integration layer, periodic review of usage and costs.

2.7 Constraints

- **Timeline.** Delivery within the Oct 2025–Jun 2026 academic schedule and stated milestones.
- **Budget.** No dedicated funding; rely on free/low-cost tiers and university resources.
- **Team capacity.** Single developer; scope must remain MVP-sized with incremental releases.
- **Scope limits.** No wearables integration, Elo-like rank systems, or outdoor crag support in the MVP.
- **Compliance.** Must satisfy App Store/Play UGC rules and GDPR (consent, minimal PII, deletion/export).

- **Content/IP and venue rules.** Uploaders must own rights; respect filming policies of partner gyms.

2.8 Resources and Stakeholders

2.8.1 Resources

- **Development team:** Responsible for the overall software development process, including requirements gathering, system design, implementation, integration of features, and maintenance. The development team is also responsible for producing the required documentation.
- **Physical facilities and equipment:** Personal computer, access to computer science labs, and climbing gyms for testing.
- **Software tools:** IDEs, version control (Git), Overleaf for documentation.

2.8.2 Stakeholders

- **Project manager:** Oversees project planning, progress and execution.
- **Climbing gyms:** Provide real-world routes and feedback on system integration.
- **Climbers (end-users):** Use the application, generate content, and provide feedback.
- **Development team:** Student implementing the system, responsible for delivering the product.

3 Specification and Modeling

3.1 Requirements Analysis [13]

3.1.1 User Stories (Illustrative)

- As a climber, I want to scan a tag to open a route so that I can quickly view details and log an attempt.
- As a gym admin, I want to require verification videos for high-grade sends to keep leaderboards fair.
- As a routesetter, I want to view a route's shared feedback, so I can tweak anything needed.

3.1.2 Use Cases

Figure 3.1 presents the UML Use Case diagram for the system, showing the three primary actors and their interactions with the platform.

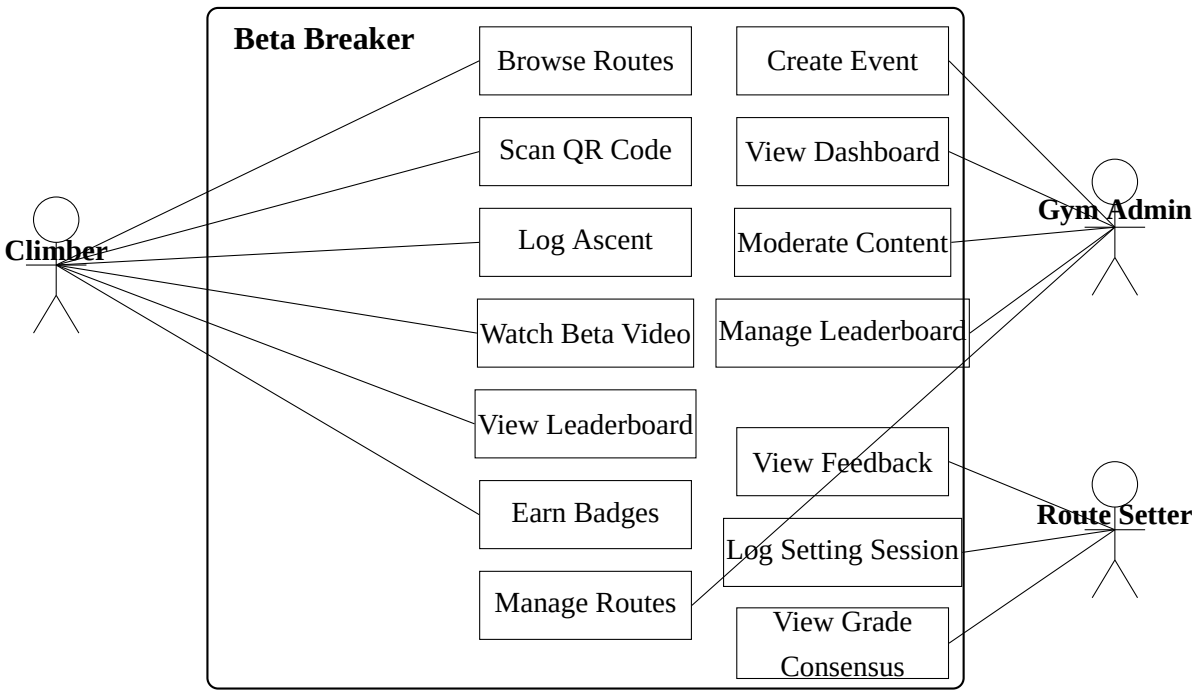


Figure 3.1: UML Use Case diagram showing the three primary actors and their interactions with the system.

3.1.3 Functional Requirements

The following multi-page table lists functional requirements by category. Each entry has an ID, a Priority Weight (PW: 5=Must, 4=Should, 3=Important, 2=Nice, 1=Future), and a brief

description.

Table 3.1: Functional requirements (by category).

ID	PW	Requirement
A. Identity & Accounts		
FR-A1	5	Email/password sign-up, login, logout, and password reset.
FR-A2	4	Profile with editable fields; user can pin up to 3 badges (Free tier limited to 1).
FR-A3	5	Account deletion and data export on request.
B. Gyms & Areas		
FR-B1	5	Gym directory with details and links to each gym's social pages.
FR-B2	2	Sectors/areas metadata and map position inside a gym.
FR-B3	4	QR/NFC station mapping to open route details from tags.
FR-B6	3	Gym logo display in directory listings, profile session cards, and leaderboard entries.
C. Problems/Routes Catalog		
FR-C1	5	Route model with setter identification; user-sourced tags aggregated by consensus.
FR-C2	4	Media attachments: beta videos.
FR-C3	5	Search & filters by grade, style, tags, recency, popularity, sent/unsent, setter.
FR-C4	4	Save routes as Project/Wishlist/Favorite and access from logbook.
FR-C5	4	System generates unique route IDs and printable QR/NFC payloads for gyms.
FR-C6	3	Feedback governance: up/down-vote tags and beta tips; low-score/spam hidden.
FR-C8	3	Video like/unlike system with like counts on route media; sort videos by most liked.
D. Tick-Logging & Sessions		
FR-D1	5	Quick log: Flash/Send, attempts count, notes, with one-tap from route or scan.
FR-D2	4	Session summary per day: attempts, sends, grade distribution.
FR-D3	4	Offline cache and sync with conflict resolution.
FR-D4	4	Explicit session start and end; compute duration.
FR-D5	2	Auto-suggest session start based on geofence/QR or multiple logs.

ID	PW	Requirement
FR-D7	4	Persistent session history with gym, duration, and linked ascent metadata; drill-down from profile logbook.
FR-D8	3	Active session hub screen: live timer, pending logs list, stats row, browse/scan action buttons, post-session summary.
E. Progression & Analytics		
FR-E1	4	Personal grade pyramid visualization.
FR-E2	3	Style insights from crowd tags (e.g., slab, overhang, dyno).
FR-E3	3	Personalized route suggestions (Pro).
FR-E4	3	Grade conversion view (Font/V/YDS) respecting gym defaults and user preference.
F. Gamification		
FR-F1	4	Badges/achievements for milestones; user can choose which to display.
FR-F2	4	Streaks (weekly/monthly) with decay and recovery mechanics.
FR-F3	3	Time-boxed challenges/quests.
FR-F4	1	Elo-like ranks (low priority), decay rules applied.
G. Social & Community		
FR-G1	4	Leaderboards by hardest grade, flash rate, volume, and rank (if enabled).
FR-G2	4	Route feedback: beta tips (text), optional beta video, user-supplied tags.
FR-G3	4	Report abuse/spam on users, videos, tags, comments (moderation queue).
FR-G4	4	Gym-configurable video requirement: for selected grade thresholds, a verification video is required for a send to count toward leaderboards; supports admin/judge review or peer threshold; missing video excludes the send from leaderboards until verified.
FR-G7	2	Leaderboard rules and prizes display: gym-configured text shown in modals on the leaderboard detail screen.
H. Competitions & Events		
FR-H1	4	Event creation (gym admin): scoring model, eligible routes.
FR-H2	4	Athlete score entry: self-entry via QR + verifier workflow or judge entry.
FR-H3	4	Live scoreboard with real-time rankings and exports for screens.
FR-H4	3	Categories (age/gender options), scheduling.
FR-H5	3	Results export (CSV/PDF) for gyms.
I. Route-Setting Workflow (Gym Admin)		

ID	PW	Requirement
FR-I1	4	Set list/calendar planning; assign setters; workload view.
FR-I2	3	Grade consensus view for setters versus community.
FR-I3	3	Maintenance tickets: report spinning/broken holds; ticket lifecycle.
FR-I4	3	Season reset: archive/rotate sets; close leaderboards per season.
J. Notifications		
FR-J1	4	Push notifications (topics configurable): friends' sends, route retirements, comp updates, rank changes.
L. Monetization & Access Control		
FR-L1	5	Roles and permissions: Climber, Gym Admin, Setter, Judge, Super-Admin (scoped per gym).
FR-L2	4	Gym billing model: €0.50 per active gym-linked user/month.
FR-L3	4	Consumer tiers: Free (1 badge; 5 beta videos/week; no analytics) vs Pro (3 badges; unlimited beta; analytics).
FR-L4	5	Pro subscription purchase flow: native IAP, restore purchases.
FR-L5	3	Trials and promo/coupon redemption for Pro.
O. Admin Web Portal		
FR-O1	4	Web dashboard for gyms: sets in progress, active routes, scan counts.
FR-O2	3	Bulk import/update (FOR REVIEW): consider replacing CSV with inline bulk editor.
FR-O3	3	Audit log: who changed grades/tags/statuses or removed media and when.
P. Data Quality & Mapping		
FR-P1	5	Grade systems: multiple systems supported with internal canonical scale.
FR-P2	4	Crowd-sourced tagging normalized into controlled taxonomy; admin alias/merge.
FR-P3	4	Anti-spoof for QR/NFC: signed payloads with rotation and client verification.
Q. Accessibility & Localization (Functional Behaviors)		
FR-Q1	3	Language switching (EN, PT-PT).
FR-Q2	2	Large text mode and larger tap targets.
FR-Q3	2	Color-aware mode (patterns/labels for hold colors).

3.1.4 Non-Functional Requirements

The following nonfunctional requirements define the expected quality attributes of the system.

- **NFR-1 – Performance and Responsiveness.** The mobile app shall load the Home screen and route details within 3 seconds on a stable 4G or Wi-Fi connection. Core actions (scanning a QR, logging a send) must provide visible feedback within 1 second.
- **NFR-2 – Availability.** Cloud services (API and database) shall maintain at least 99% uptime per month, excluding scheduled maintenance.
- **NFR-3 – Offline Operation.** Climbers shall be able to view cached routes and log attempts while offline. The app shall automatically sync new data when a connection is restored.
- **NFR-4 – Data Integrity and Security.** All communication between client and server must use HTTPS (TLS 1.2+). User credentials are encrypted and never stored in plain text. Access control is enforced according to user roles (FR-L1).
- **NFR-5 – Privacy and Data Protection.** The system shall comply with GDPR requirements, allowing users to export or delete their data (FR-A3). Sensitive media such as verification videos must respect privacy settings and consent flags.
- **NFR-6 – Scalability.** The backend shall support up to 10 000 concurrent users without degraded performance, allowing horizontal scaling through containerized deployment.
- **NFR-7 – Maintainability.** The codebase shall follow modular architecture, version control via Git, and continuous integration to enable safe incremental updates.
- **NFR-8 – Usability and Accessibility.** The interface shall follow consistent design guidelines with clear iconography and large tap targets (FR-Q2). The app must remain fully usable on common mobile screens and include a high-contrast or color-aware mode (FR-Q3).
- **NFR-9 – Reliability and Recovery.** In case of service interruption, locally stored session logs must not be lost. Sync operations shall retry automatically until confirmation of success.
- **NFR-10 – Monitoring and Logging.** The backend shall collect anonymized error and performance metrics for issue diagnosis, while excluding personal identifiers.

- **NFR-11 – Video and Media.** Beta videos shall have a maximum duration of 60 seconds and a maximum resolution of 1080p. Uploaded files must not exceed 150 MB after client-side compression. Videos are hosted on Cloudinary via unsigned upload presets. Lists use lazy-loaded thumbnails; playback uses adaptive streaming.

3.2 Modeling

3.2.1 Data Model (ER)

The data model was designed to support both the climber perspective (ascent logging, feedback, and media) and the gym perspective (route management, seasons/leaderboards, and aggregated analytics). At its core, *users*, *gyms*, and *routes* form the main domain entities. Each route belongs to a gym and stores relevant metadata (grade system/value, color, wall section, setter, and lifecycle status).

Ascent logging is represented by *route_ascents*, which associates a user to a route and stores information such as whether the route was sent, the date, rating, and optional comments. Style characterization is supported by a controlled taxonomy (*style_tags*). Users can select tags per ascent using the association table *ascent_style_tags*. These raw votes can then be aggregated into *route_style_statistics* (per-route) and *gym_style_statistics* (per-gym), enabling trend and profile analysis without recomputing everything in real time.

Motivation and community features are supported by *badges*/*user_badges* and by *leaderboards* with entries and participation snapshots. Optional *prizes* can be linked to leaderboard rankings to encourage consistent participation.

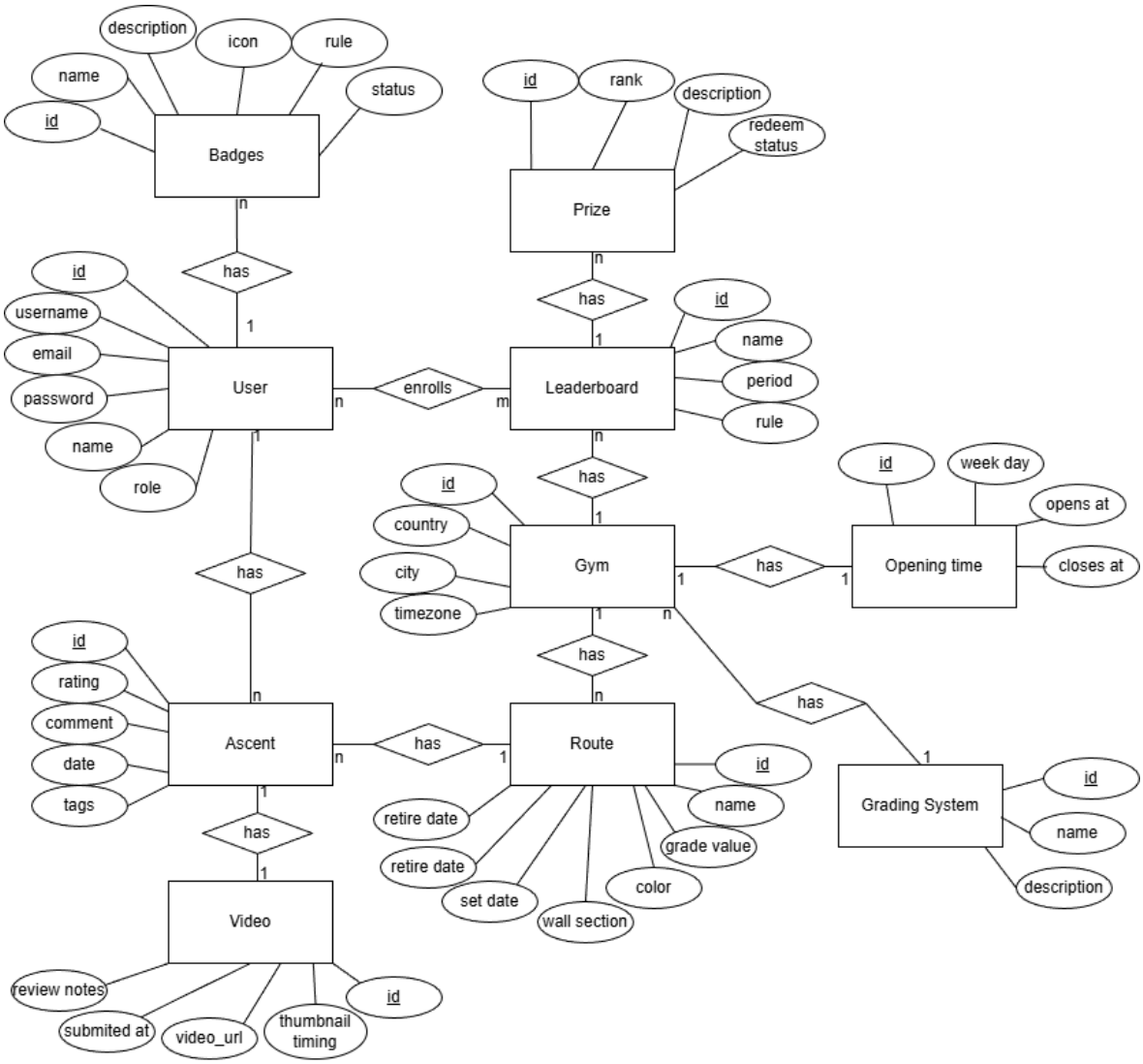


Figure 3.2: Entity–Relationship (ER) diagram of the system.

3.2.2 DB Diagram

Online diagram: dbdiagram.io/d/694574f44bbde0fd74d5a527

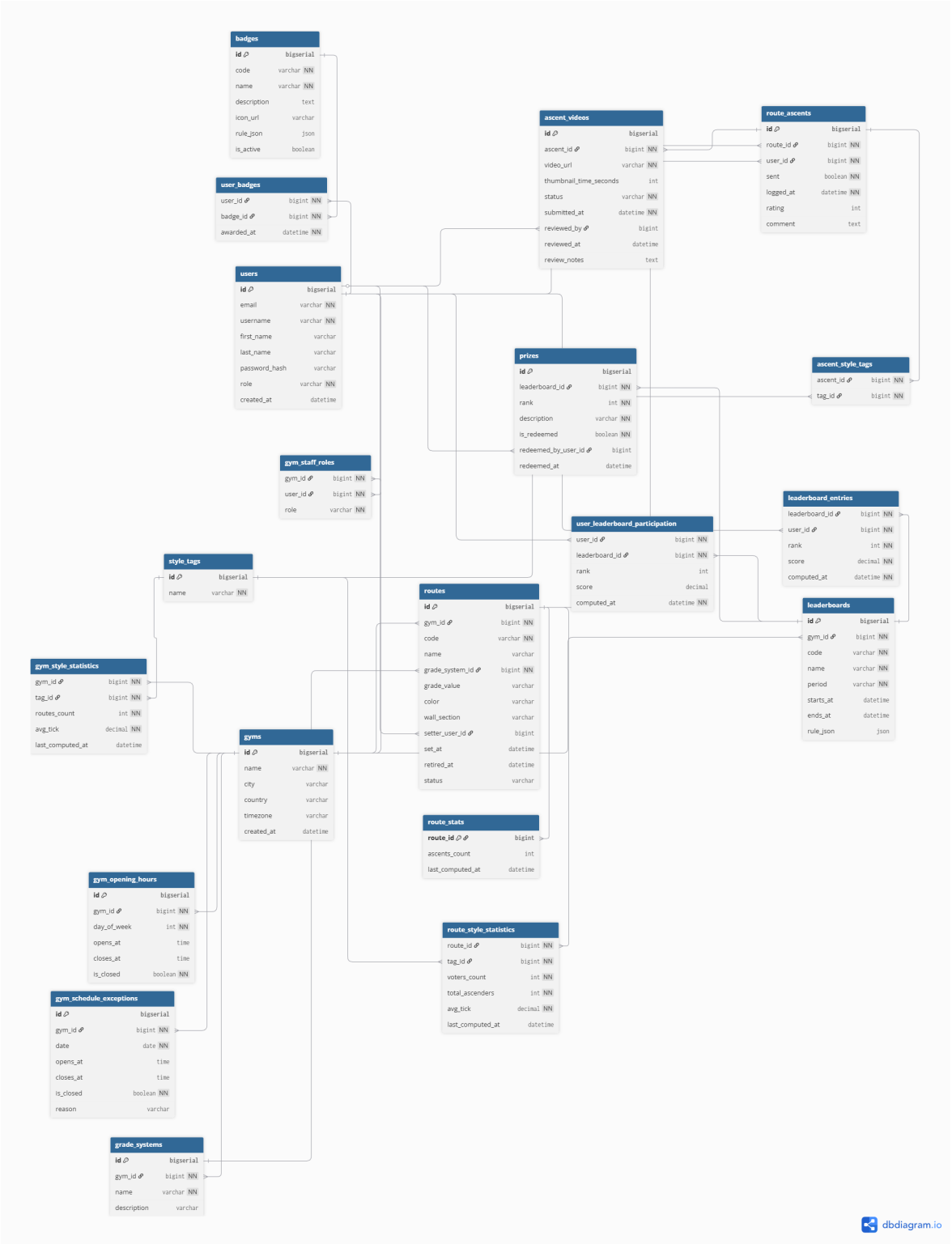
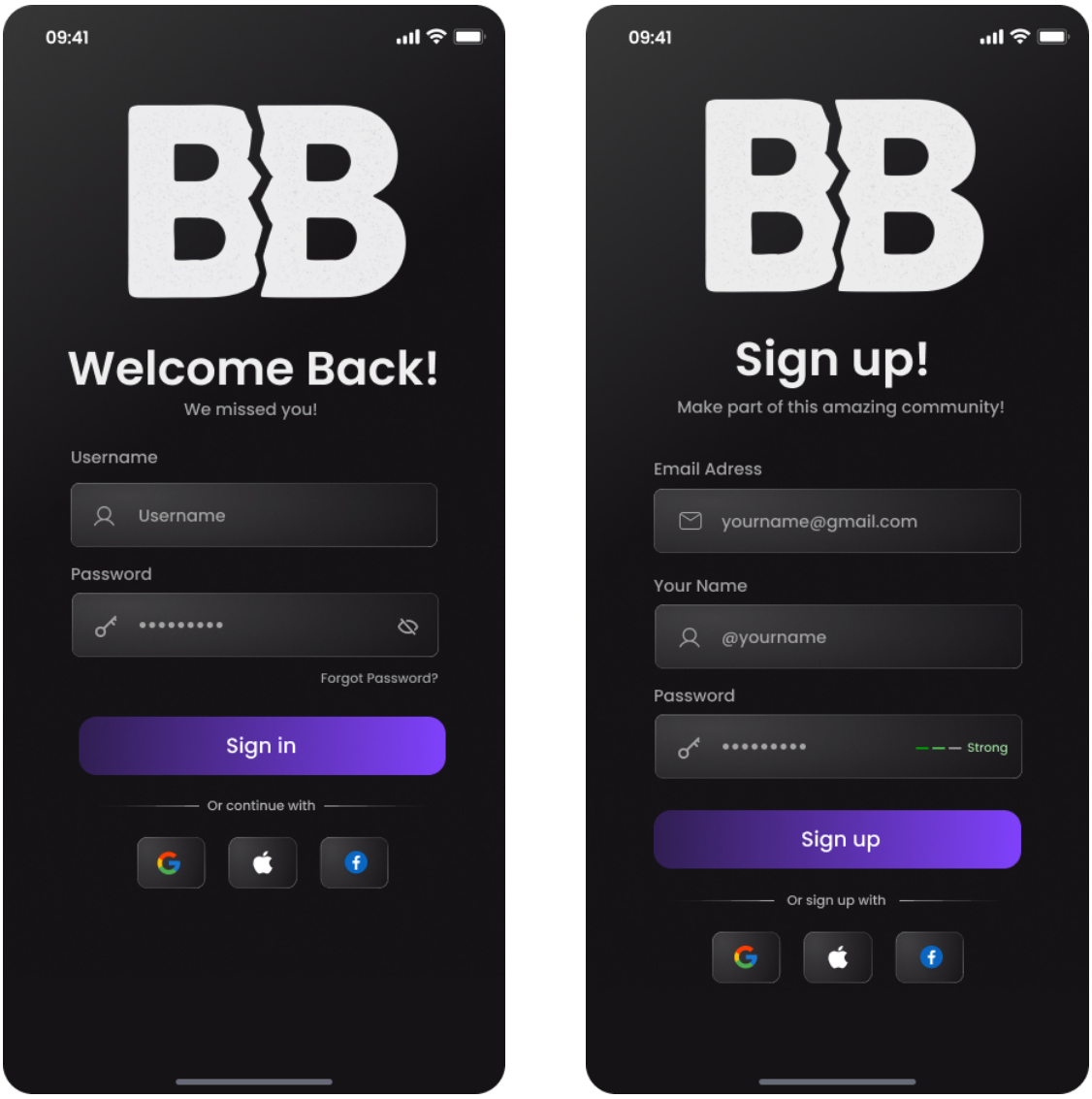


Figure 3.3: Database schema as defined in DBdiagram.

3.3 Interface Prototypes

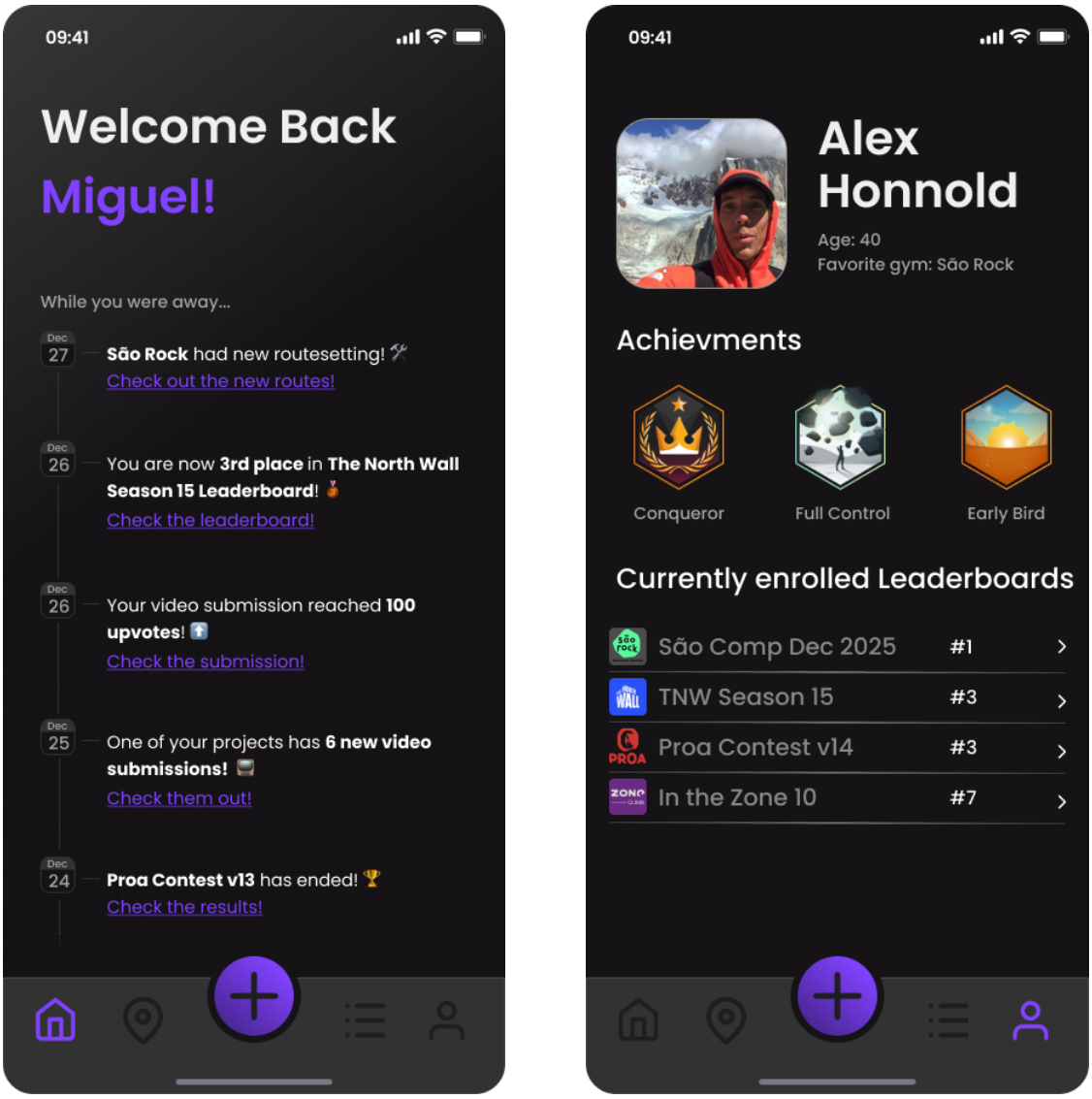
This section presents the full set of interface mockups produced for the MVP flows.



(a) Login screen.

(b) Sign-up screen.

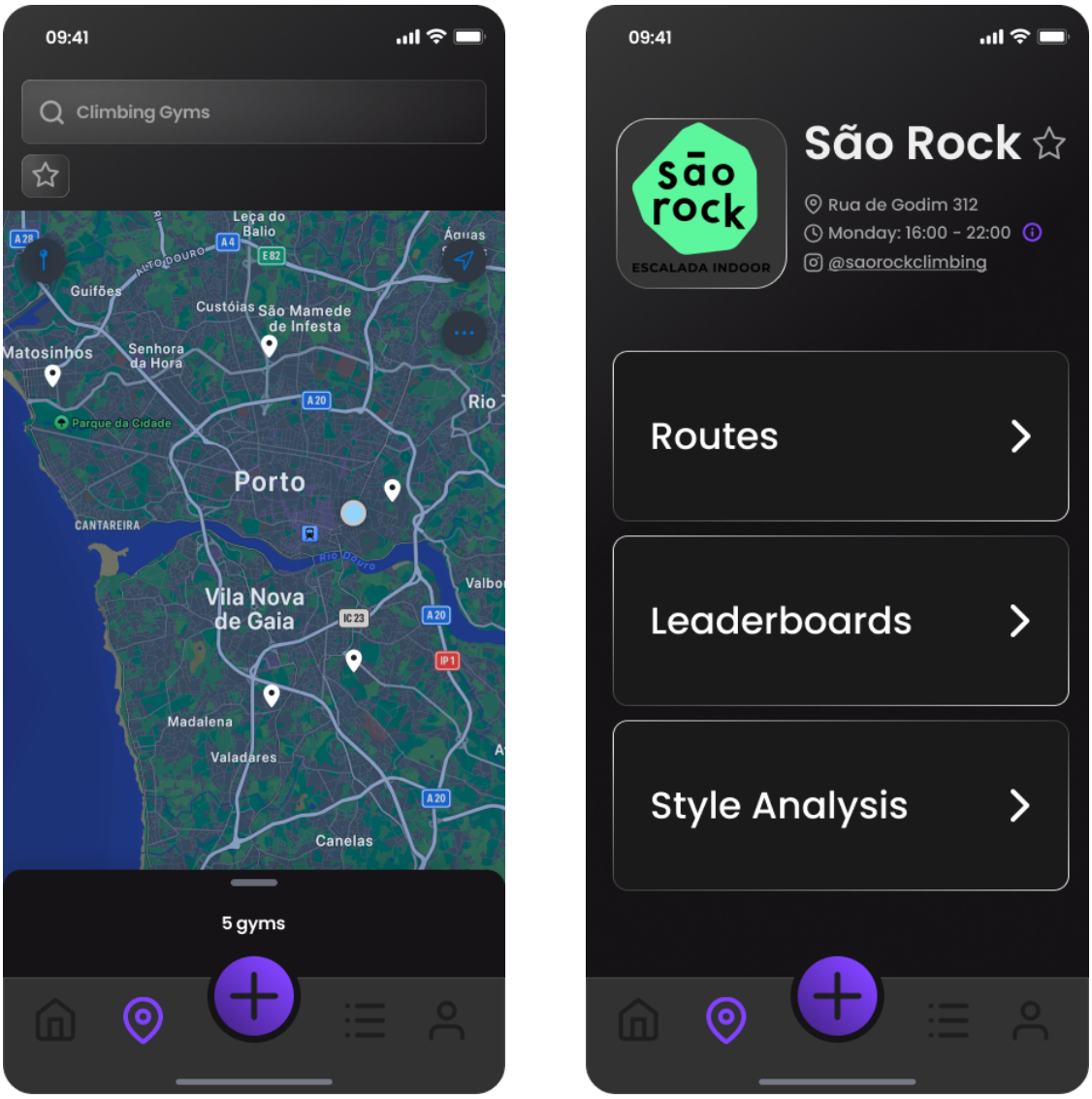
Figure 3.4: Authentication screens.



(a) Home screen.

(b) User profile screen.

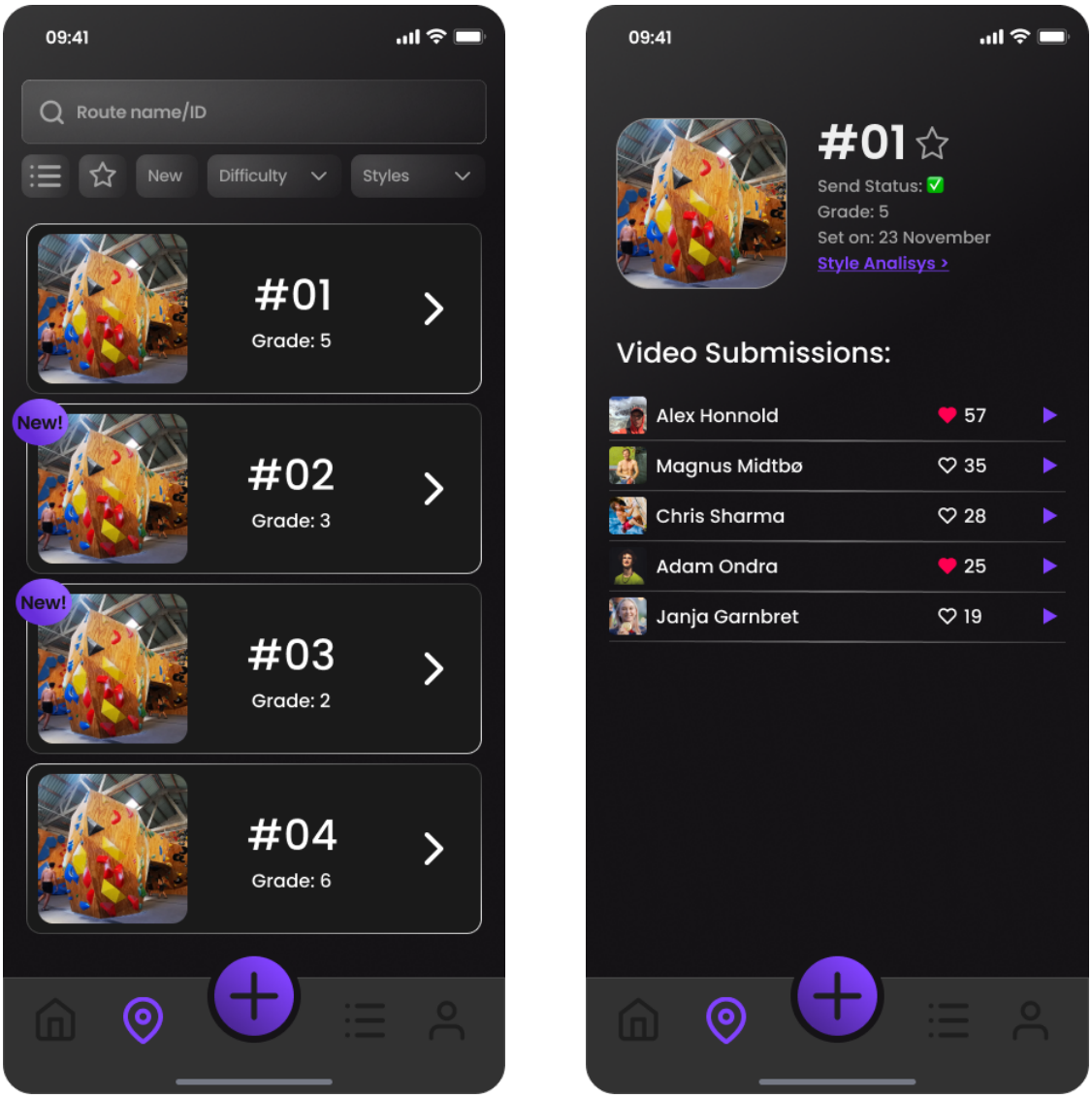
Figure 3.5: Core screens.



(a) Map browsing screen.

(b) Gym browsing screen.

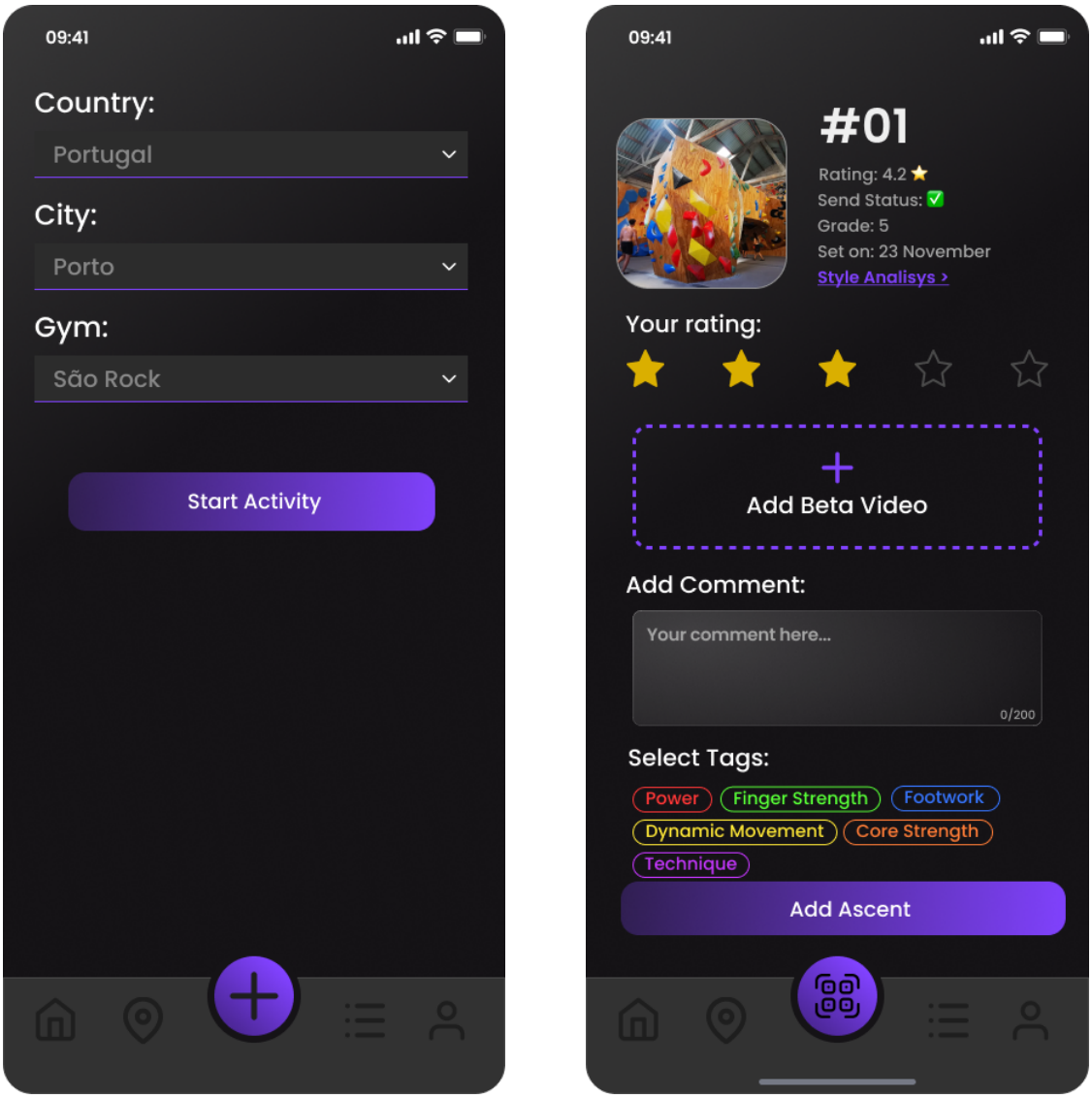
Figure 3.6: Gym discovery screens.



(a) Route browsing and filtering.

(b) Route details screen.

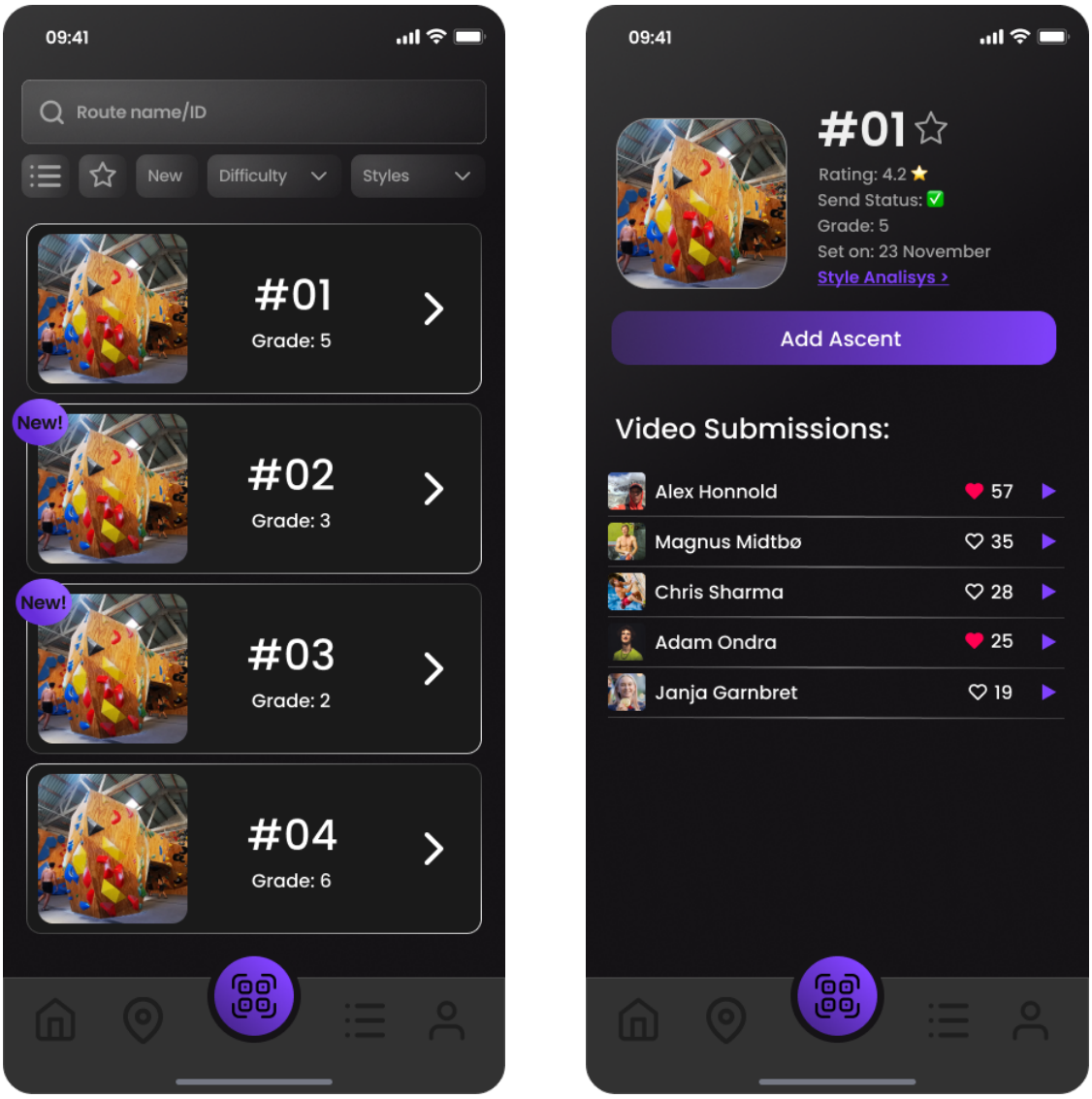
Figure 3.7: Route browsing screens.



(a) Start activity screen.

(b) Ascent logging screen.

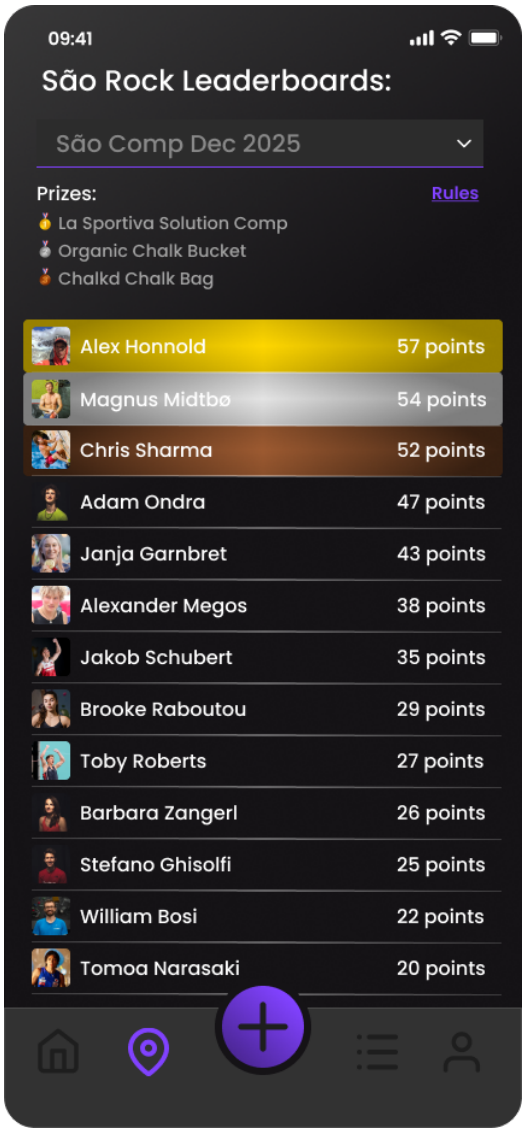
Figure 3.8: Activity and ascent logging screens.



(a) Route browsing while in an activity.

(b) Route details while in an activity.

Figure 3.9: Route screens during an activity.



(a) Leaderboard screen.

Figure 3.10: Leaderboard screen.

4 Developed Solution

4.1 Introduction

The Beta Breaker application has been implemented as a cross-platform mobile application using React Native [10] and Expo [3], backed entirely by Supabase [14] for database, authentication, storage, and real-time capabilities. As of March 2026, the project has completed 83 development steps across 20 phases (19 fully completed, Phase 8 missing only the low-priority Elo rank stretch goal), covering all planned platform features: authentication, route catalog, session logging, offline support, QR scanning, gamification, social features, notifications, competitions, beta video sharing, monetization, progression analytics, full admin portal, profile management, onboarding, internationalization (EN + PT-PT), accessibility, and Sentry error monitoring.

The implementation follows a strict Test-Driven Development (TDD) [1] methodology, resulting in 1 480 unit tests and 495 integration tests (1 975 total) across 188 test suites.

4.2 Architecture

The system is organized into two main tiers: a React Native [10] client built with Expo [3], and a Supabase [14] backend that provides the full server-side infrastructure — PostgreSQL database, authentication (GoTrue), object storage, real-time change streams, and serverless Edge Functions.

This architecture pushes authorization down to the database level [12]: every table is protected by Row Level Security (RLS) [9] policies that enforce access control based on the authenticated user's identity and role. Because RLS is evaluated by PostgreSQL itself, the security boundary cannot be bypassed by client-side code or misconfigured API routes — a stronger guarantee than middleware-based authorization in a traditional API server. The client communicates with the database through PostgREST (auto-generated REST endpoints) and with auxiliary services through five Deno-based Edge Functions: QR code signing, push notification dispatch, in-app purchase verification, billing webhooks, and promo code redemption.

State management is organized into three layers:

1. **Server state (TanStack Query v5 [15]):** All data from Supabase (routes, ascents, leaderboards, badges). The database is the source of truth; the client caches and auto-refetches.
2. **Client state (Zustand 5.x [16]):** Ephemeral UI state such as session timer, pending logs, active filters, and bottom sheet visibility. Minimal boilerplate, no providers required.

3. **Local persistence (expo-sqlite):** Offline route cache with 24-hour TTL and an offline action queue. On reconnect, actions are replayed with exponential backoff (1s, 4s, 16s).

The data flow follows four patterns:

- **Reads:** Screen → TanStack Query hook → service layer → PostgREST → Postgres (RLS-filtered).
- **Writes:** User action → Zustand optimistic update → useMutation → service → Postgres triggers (streak/badge/leaderboard) → cache invalidation.
- **Offline:** Action → offlineStore queue (SQLite) → [reconnect] → drain queue → service layer → invalidate caches.
- **Realtime:** Supabase Realtime subscription → Postgres emits changes → TanStack Query cache updated.

Figure 4.1 illustrates the high-level architecture and the relationships between the client layers and the Supabase backend services.

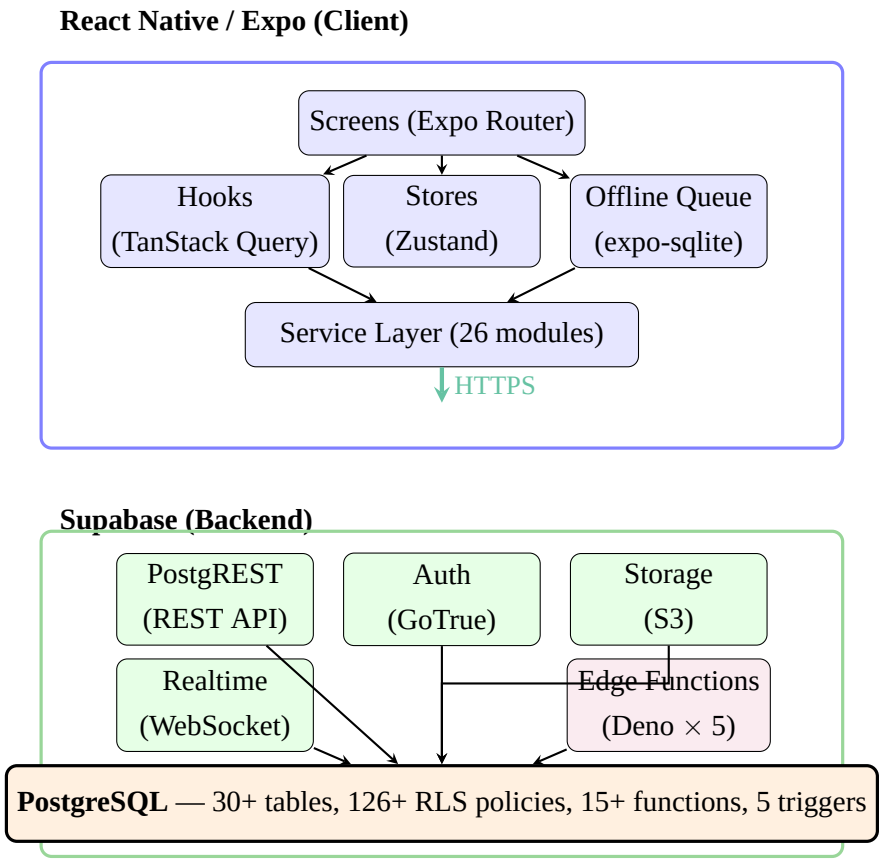


Figure 4.1: High-level system architecture. The client tier (React Native / Expo) communicates with the Supabase backend over HTTPS. Authorization is enforced at the database level by PostgreSQL Row Level Security policies.

4.3 Technologies and Tools Used

Table 4.1 lists the key technologies, their versions, and the justification for each choice.

Table 4.1: Technologies and tools used in the implementation.

Technology	**Version**	**Justification**
React Native	0.81.5	Cross-platform iOS/Android from a single TypeScript codebase.
Expo SDK	54	Managed builds (EAS), OTA updates, and native API wrappers reduce infrastructure complexity.
TypeScript	5.9	End-to-end type safety from database types to UI components.
Supabase	2.95	Postgres + RLS eliminates custom API layer. Includes managed Auth, Storage, Realtime, and Edge Functions.
TanStack Query	5.90	Battle-tested caching, optimistic mutations, and background refetch for all server state.
Zustand	5.0	Minimal boilerplate for client-only state; no providers needed.
NativeWind	4.2	Tailwind utility classes for React Native; dark-first with 25+ color tokens.
expo-sqlite	16.0	Built into Expo SDK; simple SQL interface for offline queue and route cache.
React Hook Form + Zod	7.71 / 4.3	Declarative form validation with TypeScript-first schemas.
Cloudinary	—	Video hosting via unsigned upload presets; replaces Supabase Storage for media due to React Native Blob serialization constraints.
expo-camera	17.0	QR code scanning for route identification.
expo-video	3.0	Beta video playback with lazy-loaded thumbnails.

Technology	Version	Justification
react-native-iap	15+	In-app purchase integration for Pro subscriptions.
Sentry (React Native)	latest	Error tracking with PII scrubbing and navigation performance monitoring.
i18next + react-i18next	latest	Internationalization framework for EN and PT-PT translations.
expo-notifications	0.32	Push notification delivery and token management.
Jest + RNTL	30.0 / 13.3	Unit testing with React Native Testing Library for component behavior testing.
Supabase CLI	2.75	Local Postgres instance for integration tests against real RLS policies.

4.3.1 AI-Assisted Development

Claude Code (Anthropic's CLI-based coding assistant) was used throughout the project to support both planning and implementation. All AI-generated output was reviewed and validated through the project's test suite before being integrated.

4.4 Test and Production Environments

- **Development:** Local machine with `supabase start` (Docker containers for Postgres, Auth, Storage, Realtime). Expo development server with hot reload (`npx expo start`).
- **Testing:** Jest test runner with `jest-expo` preset. Unit tests mock Supabase and native modules. Integration tests run against the local Supabase Postgres instance on port 54322.
- **CI:** GitHub Actions with three stages: lint (ESLint 9+) → type-check (`tsc --noEmit`) → test (`npm test`). Node 24.13.0.
- **Preview builds:** EAS Build with `preview` profile (iOS Simulator + Android APK).
- **Production:** EAS Build with `production` profile for store submission. OTA updates via `eas update` for JavaScript-only changes.

4.5 Scope

The following course subjects are directly applied in the implementation:

- **Databases:** PostgreSQL schema design with 30+ tables across 23 migrations, foreign keys, indexes, CHECK constraints, triggers, and stored functions. Row Level Security for authorization.
- **Software Engineering:** TDD methodology, requirements traceability, modular architecture, version control (Git), CI/CD pipeline.
- **Programming:** TypeScript, functional programming patterns (pure utility functions), object-oriented patterns (service layer), reactive programming (Realtime subscriptions).
- **Web Technologies:** REST APIs (PostgREST), JWT authentication, HTTPS/TLS, Edge Functions (serverless Deno).
- **Human-Computer Interaction:** Mobile UI design, wireframes, design system built with NativeWind [8] (dark-first, color tokens), accessibility considerations.

4.6 Components

The codebase is organized into clearly separated layers, each with a single responsibility.

4.6.1 Database Layer (23 Migrations)

The Supabase PostgreSQL database contains 30+ tables organized across 23 sequential migrations:

- **Core tables:** `profiles`, `gyms`, `routes`, `route_ascents`, `style_tags`, `route_style_tags`, `user_gym_roles`.
- **Social tables:** `route_feedback`, `route_feedback_votes`, `route_media`, `content_reports`, `follows`.
- **Gamification tables:** `badges` (19 seeded), `user_badges`, `user_streaks`, `leaderboard_entries`, `challenges`, `user_challenge_progress`.
- **Competition tables:** `events`, `event_routes`, `event_categories`, `competition_scores`.
- **System tables:** `notifications`, `push_tokens`, `notification_preferences`, `saved_routes`, `audit_log`, `maintenance_tickets`, `seasons`, `subscription_events`, `trials`, `promo_codes`, `gym_billing`, `setting_sessions`.

All tables have RLS enabled with 126 policies. Fifteen `SECURITY DEFINER` functions and five triggers handle automated logic (badge awarding, streak updates, leaderboard computation, profile creation, pinned badge enforcement).

4.6.2 Service Layer (26 Services)

Services are thin wrappers around Supabase PostgREST. They contain no business logic, only query construction:

- `auth.service.ts` — Sign-up, sign-in, sign-out, password reset, session management.
- `routes.service.ts` — Route queries with grade, style, and status filters.
- `sessions.service.ts` — Ascent logging, session summaries, session history.
- `leaderboard.service.ts` — Three scoring models (hardest grade, flash rate, volume).
- `media.service.ts` — Beta video upload to Cloudinary (unsigned preset), metadata storage, retrieval, deletion, and like/unlike mutations.
- `purchase.service.ts` — Wraps `react-native-iap` [6] for IAP purchase and receipt verification.
- Plus 20 additional services for gyms, profiles, feedback, social, events, scores, challenges, badges, notifications, moderation, saved routes, subscriptions, billing, trials, dashboard, audit, maintenance, consensus, calendar, and seasons.

4.6.3 Hook Layer (38 Hooks)

Each hook wraps a service in TanStack Query, providing caching, loading states, error handling, and cache invalidation:

- `useAuth` — Authentication state with `onAuthStateChange` listener and `refreshProfile()`.
- `useRoutes` — Route list query with infinite scroll and filter support.
- `useSubscription` — Purchase and restore mutations with receipt verification.
- `useScoreboardRealtime` — Live competition scoreboard via Supabase Realtime.
- `useOfflineSync` — Offline queue drain with exponential backoff on reconnect.
- `useAdminDashboard` — Gym-level stats: active routes, scan counts, activity feed.
- `useGradeConsensus` — Setter vs. community grade divergence data.
- `useSettingCalendar` — Route setting session scheduling and workload.
- `useProfile` — Profile editing, stats, data export, and account deletion.
- Plus 29 additional hooks for gyms, sessions, saved routes, feedback, media, social, badges, leaderboards, notifications, events, scores, challenges, admin routes, audit log, maintenance tickets, moderation, grade pyramid, style insights, route suggestions, seasons, billing, entitlements, export results, and trials.

4.6.4 Store Layer (4 Zustand Stores)

- `sessionStore` — Active climbing session state (timer, pending logs, duration).
- `offlineStore` — Offline action queue backed by expo-sqlite.
- `routeFilterStore` — Active route filter selections (grade, style, wall).
- `accessibilityStore` — User accessibility preferences (color-aware mode, screen reader hints).

4.6.5 Utility Layer (16 Pure Functions)

Pure TypeScript functions with no side effects, covering: grade conversion (V-scale, Font, YDS with canonical integer mapping 0–30), streak calculation, streak status, leaderboard scoring, competition scoring, challenge criteria evaluation, Zod validation schemas, time formatting, ISO week utilities, geolocation, QR code parsing, video validation, color name mapping, entitlement checks, personalized route suggestions, and CSV/PDF export.

4.6.6 Edge Functions (5 Deno Functions)

- `sign-qr` — Signs QR code payloads with EC P-256 keys; admin-authenticated.
- `dispatch-push` — Sends push notifications via Expo Push API with preference checks.
- `verify-iap` — Validates App Store / Play Store receipts, inserts subscription events, updates user tier.
- `billing-webhook` — Handles store renewal, cancellation, and expiration notifications.
- `redeem-promo` — Validates and applies promo codes, activates trials or Pro subscriptions.

4.6.7 UI Components (71 Components)

Components are organized by domain across 11 directories: `ui/` (10 primitives: Button, Card, Badge, BadgeIcon, Avatar, TextInput, IconButton, Divider, StarRating, ExternalLink), `routes/` (RouteCard, FilterBar, VideoUploadButton, OwnershipModal, BetaVideoPlayer), `session/` (QuickLogSheet, SessionTimer, SessionSummary), `social/` (FeedItem, FeedbackItem, FeedbackComposer, FollowButton, ReportSheet), `badges/` (BadgePicker, ProfileBadges), `streaks/` (StreakCard, StreakStatusBanner), `challenges/` (ChallengeCard, ChallengeProgress), `analytics/` (GradePyramid, StyleInsights, SuggestionsCard), `notifications/` (NotificationBell, NotificationItem), `navigation/` (CustomTabBar with FAB), `subscription/` (Paywall, PromoCodeInput), and `admin/` (CalendarGrid, GradeConsensusCard).

4.7 Interfaces

The application uses Expo Router for file-based routing with three route groups:

- `(auth)/` — Unauthenticated screens: Login, Register, Forgot Password.
- `(tabs)/` — Main app with bottom tab bar: Home, Profile, Leaderboards, QR Scanner, Logbook, Map.

- `(admin)/` — Role-gated admin screens: Dashboard, route management (CRUD, status transitions, QR), moderation queue, event management (CRUD, scoring), setting calendar, grade consensus, maintenance tickets, season management, audit log, and gym billing.

Additional screens include gym detail (`gym/[id]/`), route detail with ascent logging, event scoreboard with Realtime updates, user profiles (own and others'), notification center, settings (grade system, notification preferences, community guidelines), and an optional onboarding wizard.

5 Testing and Validation

5.1 Validation Strategy

The testing and validation plan for Beta Breaker is designed to demonstrate that the solution fulfils the functional and non-functional requirements defined in Chapter 3, operates correctly under realistic conditions, and is ready for pilot deployment with real users and gyms.

The strategy combines three complementary approaches:

1. **Automated regression testing** — Every feature is developed using Test-Driven Development (TDD): failing tests are written before the implementation, ensuring that every requirement has verifiable acceptance criteria from inception. This guards against regressions as the codebase grows.
2. **Security validation via integration tests** — Because authorization is enforced at the database level through Row Level Security (RLS) policies, correctness cannot be verified through unit tests alone. Integration tests authenticate as different user roles against a real PostgreSQL instance to confirm that data isolation holds.
3. **User acceptance and operational validation (Projeto II)** — End-to-end device tests and structured user feedback sessions with real climbers and gym staff will validate usability, performance, and fitness for purpose in a production-like environment.

This layered approach was chosen because the project's architecture pushes critical business rules into two distinct boundaries — the database (RLS, triggers, stored functions) and the client (UI flows, offline queue, state management) — each requiring a different testing technique.

5.2 Test Levels

5.2.1 Unit Tests (1 480 tests, 168 suites)

Unit tests validate individual modules in isolation:

- **Utility functions** — Grade conversion (FR-P1), streak calculation (FR-F2), leaderboard scoring (FR-G1), competition scoring (FR-H1), Zod validation schemas (FR-D1), entitlement checks (FR-L3), and video validation (NFR-11). These are pure functions with no external dependencies.
- **Service layer** — Each service module is tested with the Supabase client mocked, verifying that the correct query builder methods are called with the expected parameters.

- **Hook layer** — TanStack Query hooks are tested with services mocked, verifying caching behavior, optimistic updates, error handling, and cache invalidation after mutations.
- **UI components** — Rendered with React Native Testing Library, asserting on visible text, user interactions (press, scroll), and conditional rendering based on props and state.

5.2.2 Integration Tests (495 tests, 20 suites)

Integration tests run against a real local Supabase instance (`supabase start`) with Docker containers for PostgreSQL, Auth (GoTrue), and Storage. Each test:

1. Creates users via the Auth API and obtains JWT tokens.
2. Authenticates as a specific role (climber, gym admin, setter, super-admin).
3. Performs operations (SELECT, INSERT, UPDATE, DELETE) and asserts that RLS policies correctly allow or deny access.

These tests cover all 126 RLS policies across 30+ tables, database triggers (badge awarding, streak updates, leaderboard recomputation), stored functions (grade consensus, profile RPCs), CASCADE delete propagation, and UNIQUE constraint enforcement.

5.2.3 End-to-End Tests (Planned — Phase 20)

Full device-level tests using Maestro or Detox will validate critical user flows: authentication, scan-to-log, ascent logging with offline sync, leaderboard updates, badge awarding, Pro subscription purchase, and admin route management. These are scheduled for Projeto II.

5.3 Tools and Infrastructure

- **Jest** [7] with `jest-expo` preset — Test runner with TypeScript support via `ts-jest`.
- **React Native Testing Library** — Behavior-driven component testing (render, press, assert text).
- **Local Supabase (Docker)** — Real PostgreSQL with Auth and RLS for integration tests, accessed at `postgresql://postgres:postgres@127.0.0.1:54322/postgres`.
- **GitHub Actions CI** — Automated pipeline: lint (ESLint) → type-check (`tsc --noEmit`) → full test suite on every push and pull request.

5.4 Requirements Traceability

Table 5.1 maps each functional requirement category from Chapter 3 to the validation techniques applied and the test counts that cover it.

Table 5.1: Test coverage by requirement category.

Category	Validation Level	Tests	Key Verification
A. Identity & Accounts	Unit + Integration	96	Auth flows, profile RLS
B. Gyms & Areas	Unit + Integration	106	Directory, favorites, map
C. Routes Catalog	Unit + Integration	106	Filters, search, media, tags
D. Tick-Logging	Unit + Integration	73	Session store, summaries
E. Analytics	Unit	56	Grade pyramid, suggestions
F. Gamification	Unit + Integration	158	Badge triggers, streaks
G. Social	Unit + Integration	131	Leaderboards, feedback, reports
H. Competitions	Unit + Integration	94	Events, scoring, Realtime
I. Route-Setting	Unit	190	Calendar, consensus, tickets
J. Notifications	Unit	79	Token registration, dispatch
L. Monetization	Unit + Integration	165	IAP, billing, entitlements
O. Admin Portal	Unit	190	Dashboard, audit, moderation
P. Data Quality	Unit + Integration	49	Grade conversion, QR signing
Q. Accessibility	Unit	55	i18n, color-aware mode
Total		**1 975**	

All “Must Have” (PW=5) and “Should Have” (PW=4) requirements have corresponding automated tests. The two deferred items — FR-B2 (sector floor plans, PW=2) and FR-F4 (Elo-like ranks, PW=1) — are low-priority and do not affect core validation.

5.5 Risk and Impact Analysis

Table 5.2 presents the main risks that could compromise the correctness, security, or usability of the solution, assessed by likelihood and impact, along with the mitigation strategy and its current status.

Table 5.2: Risk and impact analysis for testing and validation.

Risk	Likelihood	Impact	Mitigation	Status
Data leakage between users (RLS misconfiguration)	Medium	Critical	495 integration tests authenticate as different roles and verify row-level isolation on every table.	Active

Risk	Likelihood	Impact	Mitigation	Status
Offline actions conflict with server state	Medium	High	Last-write-wins strategy with exponential backoff (1s, 4s, 16s). Unit tests cover queue drain, retry, and failure scenarios.	Active
Native module behavior diverges from mocks	High	Medium	Comprehensive `jest.mock()` factories simulate the native API surface. E2E tests on real devices (Phase 20) will close this gap.	Partial
Payment flow errors (IAP receipts)	Low	Critical	Edge Function validates receipts server-side against Apple/Google APIs. Integration tests verify subscription state transitions.	Active
Video upload exceeds free-tier limits	Medium	Medium	Client-side compression, 60s/150 MB caps, lazy-loaded thumbnails. Usage monitored via Cloudinary dashboard.	Active
Performance degradation under load	Medium	High	Planned: load testing with scaled seed data, Sentry performance traces, database query analysis.	Planned
Poor usability blocks adoption	Medium	High	Planned: structured user feedback sessions with pilot gym climbers and staff.	Planned

5.6 Planned Validation (Projeto II)

The following validation activities are scheduled for the second semester:

1. **End-to-end test suite (Phase 20):** Automated device tests using Maestro or Detox covering critical flows: sign-up → gym selection → route browsing → QR scan → ascent logging → leaderboard update → badge award. These tests will run on both iOS Simulator and Android emulator as part of CI.
2. **Performance and security hardening (Phase 20):** Load testing with a scaled seed dataset (1 000+ users, 500+ routes), Sentry performance traces to identify slow screens, rate-limit configuration for Edge Functions, and a security audit of all 126 RLS poli-

cies.

3. **Operational resource verification:** Validation that the production environment meets the requirements defined in Section 4.4: Supabase Pro tier ($\leq$500 MB database, $\leq$200 concurrent connections), Cloudinary free tier ($\leq$25 GB storage), and EAS Build within the 30 builds/month limit.
4. **User acceptance testing:** Pilot deployment with at least one partner gym and a group of 10–20 climbers. Structured feedback will be collected through in-app forms and individual interviews, covering usability (NFR-8), perceived performance (NFR-1), and feature completeness. Results and questionnaires will be included as an annex in the final report.

6 Method and Planning

6.1 Development Methodology

The project follows an incremental, test-driven approach [1, 13] organized into 21 sequential phases. Each phase is broken into numbered steps, and each step follows the TDD cycle:

1. **Red:** Write failing tests that define the expected behavior.
2. **Green:** Write the minimum code to make the tests pass.
3. **Refactor:** Clean up duplication, improve naming, re-run all tests.

Version control follows a trunk-based workflow on the `main` branch, with atomic commits per step. Each commit message references the step number (e.g., “feat: add QR scanner screen (Step 7.1)”). A GitHub Actions CI pipeline runs lint, type-check, and the full test suite on every push.

6.2 Milestones

Table 6.1: Project I milestones and target dates.

Event and Deliverables	**Target Date**	**Responsibility**
Project definition and proposal	October 01, 2025	Development team
Project charter approved	October 15, 2025	Development team
System requirements completed	November 05, 2025	Development team
Project plan completed	November 19, 2025[1]	Development team
Lo-fi prototype completed (wireframes and basic interactions)	December 10, 2025	Development team
Hi-fi prototype completed	January 10, 2026[2]	Development team
Prototype implementation (basic functionalities)	March 15, 2026	Development team
Gamification module (achievements + leaderboards) implemented	April 13, 2026	Development team
Testing and feedback round	May 04, 2026	Development team
Final refinements and completion of project report	May 20, 2026	Development team
Final presentation and project closure	June 03, 2026	Development team

[1] 2nd Midterm Presentation: Project Plan and Project Charter completion

[2] 3rd Midterm Presentation: Lo-fi and Hi-fi Prototypes

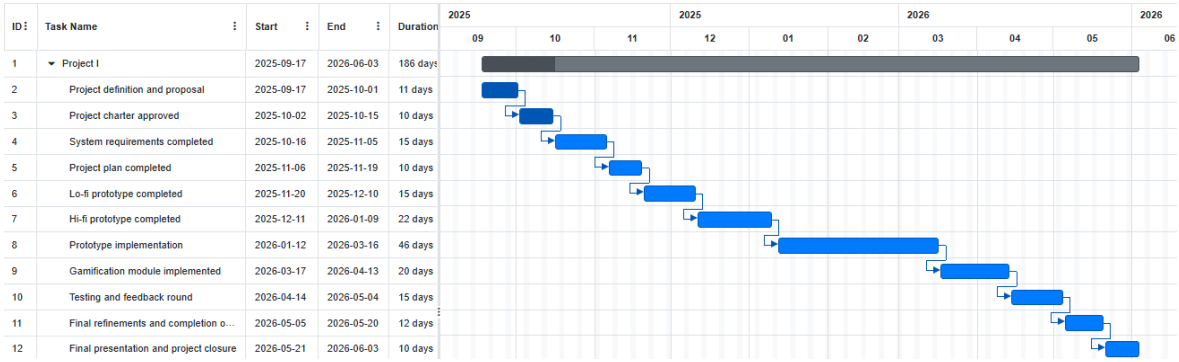


Figure 6.1: Gantt chart for Project I milestones and schedule.

6.3 Sprint Breakdown

The full development plan comprises 21 phases and 90 steps. Table 6.2 lists every step, grouped by phase, with its status and the functional requirements it addresses. Steps follow the TDD cycle: each begins with failing tests, followed by the minimum implementation, then refactoring.

Table 6.2: Complete sprint breakdown by phase and step.

Step	Description	Status	Requirements
Phase 0 — Project Scaffolding & CI Foundation			
0.1	Initialize Expo project (SDK 54, TypeScript)	Done	NFR-7
0.2	Install core dependencies	Done	NFR-7
0.3	Configure testing infrastructure (Jest, RNTL)	Done	NFR-7
0.4	Configure NativeWind (Tailwind CSS)	Done	NFR-8
0.5	Set up Supabase local dev (Docker)	Done	NFR-7
0.6	Set up directory structure	Done	NFR-7
0.7	CI pipeline (GitHub Actions)	Done	NFR-14
Phase 1 — Data Foundation (Utilities & Grade System)			
1.1	Grade conversion system (V/Font/YDS, 0–30 scale)	Done	FR-P1, FR-E4
1.2	Streak calculation logic (decay/recovery)	Done	FR-F2
1.3	Leaderboard scoring logic (3 models)	Done	FR-G1
1.4	Zod validation schemas	Done	FR-D1, FR-L1
1.5	Constants & type definitions	Done	FR-L1, FR-L3
Phase 2 — Database Schema & RLS Policies			
2.1	Core tables migration (profiles, gyms, routes, ascents)	Done	FR-A1, FR-B1, FR-C1
2.2	Row Level Security policies (role-based access)	Done	FR-L1, NFR-4
2.3	Database functions & triggers (gamification engine)	Done	FR-F1, FR-G1
2.4	Gamification tables (badges, streaks, leaderboards)	Done	FR-F1
2.5	Social & community tables (feedback, follows, reports)	Done	FR-G1–G5
2.6	Competitions & events tables	Done	FR-H1–H5
2.7	Notifications & saved routes tables	Done	FR-J1, FR-C4
2.8	Admin & audit tables (audit log, maintenance, seasons)	Done	FR-O3

Step	Description	Status	Requirements
2.9	Seed data script (19 badges, style tags)	Done	FR-F1
Phase 3 — Authentication & Session Management			
3.1	Supabase client initialization	Done	FR-A1
3.2	Auth service layer (sign-up, sign-in, password reset)	Done	FR-A1
3.3	useAuth hook (session state, onAuthStateChange)	Done	FR-A1
3.4	Design system & UI foundations (10 primitives)	Done	NFR-8
3.5	Auth screens (Login, Register, Forgot Password)	Done	FR-A1, FR-A4
3.6	Root layout & auth gate (Expo Router)	Done	FR-L1
3.7	TextInput enhancements (icons, password toggle)	Done	NFR-8
Phase 4 — Gym, Route & Tab Data Layer			
4.1	Routes service (filters, pagination)	Done	FR-C1, FR-C3
4.2	useRoutes hook (TanStack Query, infinite scroll)	Done	FR-C3
4.3	Gym service & hook	Done	FR-B1
4.4	Route Card component	Done	FR-C1
4.5	Tab bar layout with FAB (custom bottom navigation)	Done	NFR-8
4.6	Gym routes screen (list with filter bar)	Done	FR-C3
4.7	Route detail screen	Done	FR-C1
4.8	Gym main page (info, social links, sectors)	Done	FR-B1
4.9	Map browse screen	Done	FR-B1
4.10	Enrolled leaderboards tab	Done	FR-G1
4.11	Start session modal (gym selection)	Done	FR-D4
Phase 5 — Tick-Logging & Sessions			
5.1	Session store (Zustand — timer, pending logs)	Done	FR-D4
5.2	Sessions service (log ascent, summaries)	Done	FR-D1
5.3	useSession hook (mutations, cache invalidation)	Done	FR-D1
5.4	QuickLog bottom sheet (one-tap logging)	Done	FR-D1
5.5	Session timer & summary screen	Done	FR-D2, FR-D4
5.6	Logbook screen (daily history, grade distribution)	Done	FR-D2
5.7	Start activity screen	Skipped[a]	FR-D4
5.8	Full ascent form screen (notes, rating, tags)	Done	FR-D1, FR-G2
Phase 6 — Offline Support			
6.1	Offline store & SQLite action queue	Done	FR-D3, NFR-3

Step	Description	Status	Requirements
6.2	Offline route cache (24h TTL)	Done	NFR-3
6.3	Sync engine (exponential backoff: 1s, 4s, 16s)	Done	NFR-9
Phase 7 — QR/NFC Scanning			
7.1	QR scanner screen (expo-camera, JWT verification)	Done	FR-B3, FR-P3
7.2	QR signing Edge Function (EC P-256)	Done	FR-C5, FR-P3
Phase 8 — Gamification			
8.1	Badge award engine (trigger-based, SECURITY DEFINER)	Done	FR-F1
8.2	Badge display on profile (pinned badges)	Done	FR-A2
8.3	Streak UI & notifications (StreakCard, banner)	Done	FR-F2
8.4	Time-boxed challenges & quests	Done	FR-F3
8.5	Elo-like rank system (stretch goal)	Pending	FR-F4
Phase 9 — Social & Leaderboards			
9.1	Leaderboard service & hook (3 scoring models)	Done	FR-G1
9.2	Leaderboard screen	Done	FR-G1
9.3	Beta tips & route feedback (composer, voting)	Done	FR-G2
9.4	Follow system & activity feed	Done	FR-G6
9.5	Content reporting & moderation queue	Done	FR-G3
Phase 10 — Notifications			
10.1	Push token registration (expo-notifications)	Done	FR-J1
10.2	Push dispatch Edge Function (Expo Push API)	Done	FR-J1
10.3	In-app notification center (bell icon, list)	Done	FR-J1
Phase 11 — Competitions & Events			
11.1	Event CRUD (admin: create, edit, delete, categories)	Done	FR-H1, FR-H4
11.2	Score entry & verification (QR + judge workflow)	Done	FR-H2
11.3	Live scoreboard (Supabase Realtime)	Done	FR-H3
11.4	Results export (CSV/PDF with share sheet)	Done	FR-H5
Phase 12 — Media (Beta Videos)			
12.1	Video upload flow (validation, ownership modal)	Done	FR-C2, FR-K1
12.2	Video playback (lazy thumbnails, BetaVideoPlayer)	Done	FR-C2
Phase 13 — Monetization			

Step	Description	Status	Requirements
13.1	Pro subscription flow (IAP, paywall, entitlements)	Done	FR-L3, FR-L4
13.2	Trials & promo codes (redeem-promo Edge Function)	Done	FR-L5
13.3	Gym billing console (€0.50/user/month, billing-webhook)	Done	FR-L2
Phase 14 — Progression & Analytics (Pro)			
14.1	Grade pyramid visualization (Pro-gated)	Done	FR-E1
14.2	Style insights radar chart (Pro-gated)	Done	FR-E2
14.3	Personalized route suggestions (Pro-gated)	Done	FR-E3
Phase 15 — Admin Portal & Route Setting			
15.1	Admin dashboard (active routes, scan counts, activity feed)	Done	FR-O1
15.2	Route management (CRUD, status transitions, QR generation)	Done	FR-O2, FR-C5
15.3	Route setting calendar & workload view	Done	FR-I1
15.4	Grade consensus view (setter vs community divergence)	Done	FR-I2
15.5	Maintenance tickets (report + admin queue)	Done	FR-I3
15.6	Season reset & archive	Done	FR-I4
15.7	Audit log view	Done	FR-O3
Phase 16 — Profile & Settings			
16.1	Profile screen (edit, stats, export, account deletion)	Done	FR-A2, FR-A3
16.2	Other users' profile (stats, streak, badges, recent sends)	Done	FR-A2
16.3	Settings screen (grade picker, notification prefs, sign out)	Done	FR-A3
Phase 17 — Onboarding			
17.1	Optional onboarding wizard with AuthGate routing	Done	FR-A5
Phase 18 — Polish, Accessibility & i18n			
18.1	i18n setup (EN + PT-PT, i18next + react-i18next)	Done	FR-Q1
18.2	Accessibility pass (color-aware mode, screen reader support)	Done	FR-Q2, FR-Q3
Phase 19 — Error Tracking & Monitoring			

Step	Description	Status	Requirements
19.1	Sentry integration (error tracking, PII scrubbing, nav perf)	Done	NFR-10
Phase 20 — E2E Testing & Hardening			
20.1	E2E test suite (Maestro/Detox, critical flows)	Pending	NFR-1, NFR-6
20.2	Performance & security hardening	Pending	NFR-12, NFR-13
Phase 21 — Build & Deployment			
21.1	EAS Build profiles (preview + production)	Pending	NFR-2
21.2	CI/CD pipeline finalization	Pending	NFR-7
21.3	Production Supabase setup	Pending	NFR-2

[a] Step 5.7 was skipped because the existing start-session modal (Step 4.11) already satisfies the requirement; the planned screen depended on gym location columns not present in the schema.

Of the 90 planned steps, 83 are complete, 1 was skipped (Step 5.7), and 6 remain pending: Step 8.5 (Elo ranks, low-priority stretch goal) and Phases 20–21 (E2E testing and deployment).

6.4 Development Progress

As of March 2026, the project has completed 83 steps across 20 of 21 planned phases (19 fully completed, Phase 8 missing only the low-priority Elo rank stretch goal). Table 5.1 in Chapter 5 details the test coverage per requirement category.

6.5 Progress Analysis

The implementation is significantly ahead of the original milestone schedule. Core functionalities (authentication, route catalog, session logging) targeted for March 2026 were completed in early February 2026. The gamification module targeted for April 2026 was completed in February. Phases 14–19 (analytics, admin portal, profile, onboarding, i18n, accessibility, and monitoring) were all completed by early March 2026.

Key files produced: 26 service modules, 38 hooks, 71 components, 16 utility modules, 5 Edge Functions, 23 database migrations, 4 Zustand stores, 2 locale files (EN, PT-PT), and Sentry integration. The codebase consists of approximately 340 TypeScript source files (excluding tests and generated types).

Remaining phases (20–21) cover: end-to-end testing with Maestro or Detox, performance and security hardening, EAS build profiles, CI/CD finalization, and production Supabase setup.

7 Results

7.1 Test Results

The full test suite executes in approximately 52 seconds and reports:

- **Test suites:** 188 total (167 unit suites passing, 20 integration suites[1], 1 skipped[1]).
- **Unit tests:** 1 480 passing (168 suites).
- **Integration tests:** 495 (20 suites, run against local Supabase via `supabase start`).
- **Total tests:** 1 975.

All core business logic achieves high test coverage: grade conversion (11 tests), streak calculation (8 tests), leaderboard scoring (10 tests), Zod validation schemas (10 tests), entitlement checks (10 tests), competition scoring (8 tests), and route suggestions (8 tests).

Integration tests verify that all RLS policies correctly enforce row-level access across 20 test suites: users can only read their own data, service-role operations bypass RLS for administrative tasks, CASCADE deletes propagate correctly, and database functions for grade consensus, profile RPCs, setting sessions, subscriptions, trials, promo codes, and gym billing operate as specified.

7.2 Requirements Compliance

Table 7.1 maps each functional requirement category to its implementation status.

Table 7.1: Requirements compliance summary.

Category	Status	Notes
A. Identity & Accounts	Fulfilled	Email auth, profile editing, stats, data export, account deletion, onboarding wizard.
B. Gyms & Areas	Fulfilled	Gym directory with logo display (FR-B6), QR/NFC mapping, map browse. Sectors (FR-B2) deferred post-MVP.
C. Routes Catalog	Fulfilled	Route model, filters, search, media with like system (FR-C8), saved routes, style tags, admin CRUD with QR generation.

[1]The 20 integration suites require a running local Supabase instance (`supabase start`) and pass when the Docker containers are available. The 1 skipped suite (`qr-roundtrip.test.ts`) requires `supabase/.env.local` with Edge Function secrets.

Category	Status	Notes
D. Tick-Logging	Fulfilled	Quick log, session timer/summary, persistent session history (FR-D7), active session hub (FR-D8), logbook, full ascent form.
E. Analytics	Fulfilled	Grade conversion, grade pyramid, style insights radar chart, personalized route suggestions (Pro-gated).
F. Gamification	Fulfilled	19 badges, streaks with decay/recovery, time-boxed challenges. Elo ranks (FR-F4) deferred as stretch goal.
G. Social	Fulfilled	Leaderboards (3 models) with rules/prizes display (FR-G7), feedback, follows, activity feed, content reporting, moderation queue.
H. Competitions	Fulfilled	Event CRUD, score entry & verification, live scoreboard with Realtime, CSV/PDF export.
I. Route-Setting	Fulfilled	Setting calendar with workload view, grade consensus (setter vs community), maintenance tickets, season reset & archive.
J. Notifications	Fulfilled	Push token registration, dispatch Edge Function, in-app notification center.
L. Monetization	Fulfilled	Pro subscription IAP flow, entitlements, paywall, trials, promo code redemption, gym billing console.
O. Admin Portal	Fulfilled	Dashboard with stats and activity feed, route management, moderation queue, event management, audit log.
P. Data Quality	Fulfilled	Grade systems with canonical scale, crowd-sourced tagging, QR anti-spoof (signed JWT).
Q. Accessibility	Fulfilled	Language switching (EN, PT-PT via i18next), color-aware mode, screen reader support, accessibility store.

All 14 requirement categories are fulfilled. The only deferred items are FR-B2 (sector floor plans, post-MVP) and FR-F4 (Elo-like ranks, low-priority stretch goal). All “Must Have” (PW=5) and “Should Have” (PW=4) requirements are implemented.

8 Conclusion

8.1 Conclusion

This report documents the specification and implementation of Beta Breaker, a mobile platform for indoor climbing gyms. The project addressed the identified problem — fragmented route knowledge and lack of structured gym-climber feedback — by building a comprehensive system that connects climbers and gyms through route discovery, ascent logging, beta sharing, and gamification.

The implementation followed a strict TDD methodology across 20 phases and 83 steps, producing 1 975 automated tests (1 480 unit + 495 integration). The system covers all planned features: authentication, route catalog, session logging, offline support, QR scanning, gamification (19 badges, streaks, challenges), social features (leaderboards, follows, feedback, moderation), push notifications, competitions with live scoreboards, beta video sharing, Pro subscription monetization via in-app purchases with trials and promo codes, gym billing, progression analytics (grade pyramid, style insights, personalized suggestions), a full admin portal (dashboard, route management, setting calendar, grade consensus, maintenance tickets, season management, audit log), profile management, onboarding, internationalization (EN + PT-PT), accessibility (color-aware mode, screen reader support), and Sentry error monitoring.

The architecture — Supabase-first with no custom backend, TanStack Query for server state, Zustand for client state, and expo-sqlite for offline persistence — proved effective for a single-developer project, minimizing infrastructure complexity while maintaining security through database-level RLS policies.

8.2 Future Work

The remaining work consists of two final phases:

1. **Phase 20 — End-to-End Testing & Hardening:** E2E test suite (Maestro or Detox) covering critical user flows (auth, scan-to-log, offline sync, leaderboard update, badge award, competition, admin route lifecycle, season reset, Pro subscription). Performance profiling with Sentry traces, rate limit configuration, load testing with scaled seed data, and security audit of RLS policies.
2. **Phase 21 — Build & Deployment:** EAS Build profiles (development, preview, production), CI/CD pipeline finalization (lint $\rightarrow$ type-check $\rightarrow$ test $\rightarrow$ deploy), production Supabase project setup (migrations, storage buckets, Realtime, Edge Function secrets), and store submission.

Additionally, one low-priority stretch goal remains: Step 8.5 (Elo-like rank system with decay rules, FR-F4, PW=1).

Completing these phases will result in a production-ready platform deployable to the App Store and Google Play, ready for pilot testing with partner climbing gyms.

Bibliography

[1] K. Beck, *Test-Driven Development: By Example*, Addison-Wesley, 2003. (livro)

[2] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, “From Game Design Elements to Gamefulness: Defining Gamification,” in *Proc. 15th International Academic MindTrek Conference*, pp. 9–15, 2011. (artigo conferência)

[3] Expo Documentation, *Expo SDK 54 Reference*, 2025. `https://docs.expo.dev/` (website)

[4] J. Hamari, J. Koivisto, and H. Sarsa, “Does Gamification Work? — A Literature Review of Empirical Studies on Gamification,” in *Proc. 47th Hawaii International Conference on System Sciences (HICSS)*, pp. 3025–3034, 2014. (artigo conferência)

[5] i18next Contributors, *i18next Internationalization Framework Documentation*, 2025. `https://www.i18next.com/` (website)

[6] react-native-iap Contributors, *react-native-iap v15+ Documentation*, 2025. `https://react-native-iap.dooboolab.com/` (website)

[7] Jest Contributors, *Jest Testing Framework Documentation*, 2025. `https://jestjs.io/docs/getting-started` (website)

[8] NativeWind Contributors, *NativeWind v4 Documentation*, 2025. `https://www.nativewind.dev/` (website)

[9] PostgreSQL Global Development Group, *PostgreSQL 15 Documentation — Row Security Policies*, 2025. `https://www.postgresql.org/docs/15/ddl-rowsecurity.html` (website)

[10] Meta, *React Native Documentation*, 2025. `https://reactnative.dev/docs/getting-started` (website)

[11] Sentry Contributors, *Sentry for React Native Documentation*, 2025. `https://docs.sentry.io/platforms/react-native/` (website)

[12] A. Silberschatz, H. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill, 2019. (livro)

[13] I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2015. (livro)

[14] Supabase Documentation, *Supabase Platform Guide*, 2025. `https://supabase.com/docs` (website)

[15] TanStack, *TanStack Query v5 Documentation*, 2025. `https://tanstack.com/query/latest` (website)

[16] Zustand Contributors, *Zustand State Management Documentation*, 2025. `https://zustand.docs.pmnd.rs/` (website)